

ctms-postman-collections (IPWS TO CTMS for Avid Production Management)

Name

Avid API Tooling - **Avid Production Management**
Postman Collections [Avid's official Documentation](#)

Description

This Git repository contains a comprehensive set of examples designed to assist with the transition from **IPWS** to **CTMS** within a **Avid Production Management** system environment. Each folder includes curated Postman collections that can be imported into your Postman application, showcasing a step-by-step walkthrough of various **CTMS** API calls integrated with your on-premise system.

These examples serve as a valuable reference to deepen your understanding of **CTMS** functionality and internal workflows. They are especially useful for developers and architects who are building or designing applications or features that interact with **CTMS**.

Usage

Leverage these examples to explore how different **link relations** and **features** are implemented and utilized within **CTMS**. Each folder includes documentation to guide you through the intended use cases and the reasoning behind each step. Every collection typically follows a structured flow:

1. Initiating with **user authentication** and **token retrieval**
2. Executing a series of interconnected **API** calls relevant to a particular scenario
3. Concluding with a **user logout** operation

Users are encouraged to review the in-folder documentation to gain a clearer understanding of the flow and expected outcomes at each stage.

How to Start with These Collections

To begin using the **CTMS** Postman collections provided in this repository, follow the steps below to import them into your Postman application and start interacting with your system.

Step 1: Download the Collections

Navigate to the respective folders in this repository and download the Postman collection files (`.postman_collection.json`). Each folder contains one or more collections demonstrating a specific set of use cases or **API** workflows.

Step 2: Import into Postman

1. Open the **Postman** application.
2. Click on the **"Collections"** tab in the left sidebar.
3. Click the **"Import"** button (top-left or via File > Import).
4. Select the desired `.postman_collection.json` file from your local system.
5. Once imported, the collection will appear in your **Collections** sidebar.

Step 3: Select the Environment

Before running any collection, make sure to:

- Import the provided **Postman environment JSON** file.
- Configure the required variables (e.g., `mcs_user`, `mcs_password`, `mcs_host`).
- Configure the system you want to play with (insert appropriate values in `system_type` and `system_name`), you can refer your CloudUX application to get these values.
- Reference [Importing and Configuring Postman Environment](#) for detailed instructions.

Step 4: Run the Authentication(Login) Collection

1. Expand the imported collection in the sidebar and look for **"Authentication Login Logout"** folder.
2. Run the `MCS get identity providers` Request.
3. Move to the `MCS ROPC login` and in **query paramter** section, add your Authorization bearer token in the `Authorization` key.
4. Run the ROPC login request now to authenticate yourself.

Step 5: Run the Collection of your choice

1. Look for the collection of your choice in the given system.

2. Follow the documentation inside the folder or collection to understand the sequence of **API** calls.
 3. Observe the request and response flow to understand how **CTMS** works behind the scenes.
-

Expectations from You as a User

Before using these Postman collections, it is expected that users have a **clear understanding of the functionality they are exploring** and the implications of executing specific **API** requests. As these collections interact directly with your on-premise system, please ensure the following precautions are taken:

1. **Environment Setup:**
Import the provided Postman environment and populate the required fields — **mcs_host**, **mcs_user**, and **mcs_password** — with values relevant to your system configuration.
2. **System-Specific Variables:**
Fields such as **assetIDs**, **folderIDs**, **asset_path**, etc., are highly dependent on your individual system setup.
 - ♦ *You are responsible for identifying and using appropriate values for these fields.*We recommend retrieving such information from **CloudUX** or by invoking the **CTMS Service Provider** API in Postman.
3. **Caution with Destructive Operations:**
Exercise caution when executing operations like **deleting assets** or **creating reservations**. Ensure you are referencing the correct resources to avoid unintended data loss or system modification.
4. **Recommended Practice:**
Create a **dedicated test folder** in your environment to serve as a **sandbox** for safely exploring these **API** calls.
You may use this folder to create new assets or duplicate existing ones.
Use the IDs of these test assets for experimentation to avoid impacting production data.

 *To fetch the ID of an asset using CloudUX:*

Go to **CloudUX**, move to the folder, click on the desired asset, which will open a side panel with an asset player.

Go to the **Metadata** tab of this panel, scroll down to find the asset ID as the **mobID**.

User Inputs

- Before using any collection, make sure you have correct values set up in your **environment variables**, specially for `mcs_host`, `mcs_user`, `mcs_password`. Input correct values for `system_type` and `system_name` associated to the system you want to test these collections on.
- During the execution of these Postman collections, users are expected to provide input values — such as `assetIDs`, `folderIDs`, `masterClipIDs`, etc. — by setting them as **environment variables**.
- Wherever user input is required, it will be **clearly indicated** in the collection-specific documentation in the following format:
 - ◆ At this step, you need to insert the appropriate Asset ID in the environment variable named: `assetID`

Please ensure that the values provided correspond to **valid entities** within your system to avoid errors or unintended behavior.

Prerequisites of Running Any Collection

-  Select the correct environment in Postman.
-  If not already imported, import the `default-environment` from the path: `/environment/`
-  Ensure that valid credentials are set for the variables `mcs_user` and `mcs_password`.
-  Select the correct `system_type` and `system_name` values, you can refer to your cloudUX app for these values.

Authentication as a User

Every collection contains the following two essential API calls to authenticate the user:

GET MCS get identity provider → sets `mcs_identity_provider_url`

- This GET call retrieves the identity provider URL for the given host.

- In the **Scripts** section: It extracts the `"auth:identity-provider" > "auth:ropc-default"` URL and stores it in the global variable `mcs_identity_provider_url`.

POST MCS ROPC login

- This POST call authenticates using ROPC credentials.
- In the **Body**: You must provide `username`, `password`, `grant_type`, and `scope`.
- In the **Scripts**: The response includes access tokens. These are stored as global variables:
 - `avidAccessToken` – used for Bearer authentication
 - `AvidCookie` – used for cookie-based authentication in subsequent requests

Support

If you encounter issues or have questions regarding the execution or structure of a collection, please:

1. Review the **Recommendations** section in the documentation.
2. Ensure all environment variables are correctly populated.
3. If the issue persists, reach out to the **Avid Support Team** for further assistance.

Authors and Acknowledgment

The following `CTMS` link relations have been developed by the **MediaCentral | Serenity** team. These Postman collections are curated and maintained by the **API Tooling Team**.

License

Any and all usage of these collections is **licensed** and subject to the terms defined by Avid.



Importing and Configuring Postman Environment

To ensure a smooth and secure experience while using the provided Postman collections, follow the steps below to import and configure the Postman environment JSON file in your Postman application.



Step 1: Import the Environment File

1. Open the **Postman** application.
 2. In the left sidebar, click on the **"Environments"** tab (accessible via the gear icon  at the top-right).
 3. Click on the **"Import"** button.
 4. Select the provided `default-env.postman_environment.json` file from your system.
 5. Once imported, you will see the environment listed in your environments' dropdown.
-



Step 2: Configure Required Variables

The imported environment contains a predefined set of variables required to interact with your on-premise system. These include:

- `mcs_user`
- `mcs_password`
- `mcs_host`



These variables are initially left empty. You must populate them before executing any collection.

Instructions:

1. Click on the imported environment.
2. Enter your system's specific values for:
 - `mcs_user`: Your username
 - `mcs_password`: Your password
 - `mcs_host`: Hostname or IP address of your CloudUX server
3. Save the environment.



Other Environment Variables

The environment also contains additional variables such as:

- `asset_id`
- `folder_id`
- `master_clip_id`
- `folder_path`
- `asset_path`, etc.

These are **system-specific** values and must be updated manually based on your available assets and folders.

You can discover these IDs using:

- The **CloudUX interface**
 - Postman API calls (e.g., `CTMS Service Provider` collection)
- ◆ The collection-specific documentation will indicate exactly where and which variable you need to populate.
For example:
"At this step, you need to insert the appropriate Asset ID in the environment variable named: `asset_id`"



Recommended Practice

To ensure a safe and controlled environment:

- **Create a dedicated test folder** in your system.
- Use this folder to create or duplicate assets for testing purposes.
- Reference the IDs of these assets/folders when running destructive operations (e.g., delete, update).

This minimizes the risk of accidentally modifying or deleting production content.



Important Cautions

- **Do not run collections before configuring `mcs_user`, `mcs_password`, and `mcs_host`** — authentication will fail.
 - Avoid using production asset/folder IDs unless you are certain of the operation.
 - Always review pre-filled variables before executing operations such as delete, reserve, or modify.
-

By properly configuring your environment and understanding the responsibilities around variable management, you ensure secure and accurate execution of CTMS API workflows within your own system.

Documentation Title: Add Asset to Folder (LinkToMob)

Summary:

The Interplay Web Services method LinkToMOB adds a reference to an asset, identified by its MOB ID, to a folder.

Add an asset to a folder in CTMS

Parameters:

- The MOB ID of the asset.
- The ID of the target folder, consisting of the path with leading and trailing slashes.
Example: `"/Projects/Example/"`.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [loc:item-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the target folder and expand the URI template.
2. Make an HTTP GET request to get the [loc:item](#) resource of the folder.

Make an HTTP POST request to the [loc:add-item](#) link with a JSON object with a property "base" with sub properties "systemType" (the Avid Production Management type "avid-pmplus"), "systemID" (the ID of the Avid Production Management), and "id" (the MOB ID of the asset to add) as body:

None

```
{
  "base": {
    "systemType": "avid-pmplus",
    "systemID": "DD481E2E-260B-4684-9D25-44C774E3F782",
    "id":
    "060a2b340101010101010f0013-000000-070812007d070018-060e2b347f7f-d080"
  }
}
```

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Add asset to Folder (LinkToMob)/CTMS Add asset to folder (LinkToMob).postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_locItemByIdHref`.

◆ At this step, you need to insert the appropriate **folder ID** in the environment variable

named: folderIdInput

GET 2_loc item resource of folder

- Hitting the above fetched selected_system_locItemByIdHref, to get the location/folder structure details of the selected folder ID.
- In the **Scripts** section: We fetch the "[loc:add-item](#)" link from the _links in the HAL output, and store it in selected_system_locAddItemHref variable globally.

POST 3 add item

- We hit the selected_system_locAddItemHref link from above, that helps us to add the given asset to a given folder path.
- In the **Body** section: Add the asset information from "base" and "id" in the request body and send it along with the add item URL.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Attributes

Get attributes of assets in CTMS

Summary:

The Interplay Web Services method GetAttributes returns user and system attributes for assets and folders.

Parameter:

- The MOB ID of the asset.
- Optional: a list of attributes to request.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link aa:asset-by-id for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the masterclip and expand the URI template.
 2. Add the query parameter "embed=attributes".
 3. Optional: add the query parameter "attributes=a,b,c" with a comma-separated list of attributes to request. If not given, a default set of attributes is returned.
 4. Make an HTTP GET request to get the aa:asset resource with an embedded aa:attributes resource.
3. Evaluate the "common" property of the aa:asset resource and the "attributes" property of the aa:attributes resource. See below to see how to map IPWS attributes to CTMS attributes.

You can also use the link aa:asset-by-id-bulk to query multiple assets with a single bulk call. Avid recommends keeping the number of assets per request small (<= 20).

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Attributes/CTMS Get Attributes of Assets User Input.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as selected_system_aaAssetByldHref.

◆ At this step, you need to insert the appropriate **asset Id** in the environment variable named: assetId

GET 2_asset by id

- Hitting the above fetched selected_system_aaAssetByldHref, to get the details of the given asset.
- In the **Scripts** section:

- We fetch aa:asset link's href, present inside ["_embedded"]["aa:attributes"]["_links"], and store it globally as selected_system_aaAssetHref.
- We fetch aa:attributes link's href, present inside ["_links"], and store it globally as selected_system_aaAttributesHref.

GET 3.1_check attributes at aa:asset href: common property

- Hitting the above selected_system_aaAssetHref command, to verify the attributes output associated to the given asset as shown above in summary.

GET 3.2_check attributes at aa:attributes href: attributes property

- Hitting the above selected_system_aaAttributesHref command, to verify the similar output in the attributes section of the output.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Get attributes of a folder

Parameters:

- The ID of the folder, consisting of the path with leading and trailing slashes. Example: "/Projects/Example/".
- Optional: a list of attributes to request.

Steps:

1. Query the information from the CTMS Registry.
2. Find the link loc:item-by-id for the Avid Production Management. The "href" is an URI template.
 1. Set the URI template variable "id" to the ID of the folder and expand the URI template.
 2. Add query parameter "?attributes=*" to get all attributes, or "?attributes=a,b,c" to get a specific list of attributes.
 3. Make an HTTP GET request to get the loc:item resource of the folder.
3. Evaluate the properties "common" and "attributes".

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Attributes/CTMS Get Attributes of Folder User Input.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_locItemByIdHref`.

◆ At this step, you need to insert the appropriate **folder Id** in the environment variable named: `folderIdInput`

GET 2_loc item by id for the folder

- In the **Query Params** section: Add key as `attributes` and value as `*`.
- Hitting the above fetched `selected_system_locItemByIdHref`, to get the folder structure details of the given folder id.
- In the **Scripts** section: We fetch the "[aa:attributes](#)" link present inside the `_links`, and store it globally as `selected_system_aaAttributesHref`.

GET 3_check attributes at `aa:attributes href: attributes` property

- Hitting the above fetched `selected_system_aaAttributesHref`, to get the attributes property of the given folder.

Precautions/Recommendations:

- Please **select and put** the right asset/folder IDs in the environment required for this example.

Set attributes of assets in CTMS

- The Interplay Web Services method `SetAttributes` updates user and system attributes for assets and folders.

Parameters:

- The MOB ID of the asset.
- An array of attributes to update.

Avid recommends updating the attributes via an embedded aa:attributes resource as shown below. You can update a few common asset attributes directly but there are tight restrictions for that.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link aa:update-asset-by-id for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the masterclip and expand the URI template.
2. Make an HTTP PATCH request with an aa:asset resource with an embedded aa:attributes resource with the attributes to set as request body. See below for the mapping between IPWS and CTMS. Example:

None

```
{
  "_embedded": {
    "aa:attributes": {
      "attributes": [
        {
          "name": "com.avid.workgroup.Property.User.IPWSTest",
          "value": "The weather is beautiful outside."
        },
        {
          "name": "Comments",
          "value": "Weather report"
        }
      ]
    }
  }
}
```

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Attributes/CTMS Set Attributes of an Asset.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:update-asset-by-id](#)" link present inside the "resources", and store the variable globally as selected_system_aaUpdateAssetByIdHref.

◆ At this step, you need to insert the appropriate **asset Id** in the environment variable named: masterClipId

PATCH 2_Update Asset by ID

- In the **Body** section, add the JSON for embedded attributes, as shown above in the summary tab.
- Hitting the above fetched selected_system_aaUpdateAssetByIdHref, to update the given asset ID.

Precautions/Recommendations:

- Please **select and put** the right asset/folder IDs in the environment required for this example.

Documentation Title: Categories

Set categories for an asset in CTMS

Summary:

The Interplay Web Services method SetCategories sets the categories for assets and folders.

The list of available categories is pre-configured in the MediaCentral | Avid Production Management. See "Get categories via CTMS" to learn how to query the list of configured categories. CTMS replaces the categories of the asset with the given array of categories. The IPWS method works differently and accepts an incremental update with categories to add and to remove. If you want to replace that semantic, you must first get the current categories, add or remove categories in the list, and then use the CTMS link below to replace the categories with the resulting list.

Parameter:

- The MOB ID of the asset.
- An array of categories.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [aa:update-asset-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the masterclip and expand the URI template.
2. Make an HTTP PATCH request with an [aa:asset](#) resource with an embedded [aa:attributes](#) resource with the attribute "com.avid.workgroup.Property.Category" in the request body. The value of the attribute is an array of entries, each with an increasing row number in the property "index", and the category to set in the property "value".

None

```
{
  "_embedded": {
    "aa:attributes": {
      "attributes": [
        {
          "name": "com.avid.workgroup.Property.Category",
          "value": [
            {
              "index": 0,
              "value": "category1"
            }
          ]
        }
      ]
    }
  }
}
```

```

    },
    {
      "index": 1,
      "value": "category2"
    }
  ]
}
]
}
}
}

```

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Categories/CTMS Set Categories of an Asset.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:update-asset-by-id](#)" link present inside the "resources", and store the variable globally as selected_system_aaUpdateAssetByIdHref.

◆ At this step, you need to insert the appropriate **master clip ID** in the environment variable named: masterClipId

PATCH 2_update aa asset by id

- Hitting the above fetched `selected_system_aaUpdateAssetByIdHref`, to get the response associated to the asset, showing the associated categories.
- In the **Body** section: Add the request body, as mentioned above in the summary tab.

Please make sure, that you have set the categories "category1", "category2" etc. in your system before running this.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.
- The categories must be set beforehand, otherwise, the system will throw an error if unknown values of categories is sent in the request body.

Get categories for an asset in CTMS

The Interplay Web Services method `GetCategories` can be used for two purposes:

- Get the list of categories that are configured for the Avid Production Management.
- Get the categories that are assigned to a concrete asset or folder.

Get configured categories for the Avid Production Management in CTMS

Steps:

Use the cached CTMS Registry information or fetch it as shown in [Using the CTMS API](#).

Find the link [taxonomies:taxonomy-by-taxonomyid](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "taxonomyid" to "com.avid.workgroup.Property.Category" and expand the URI template.
2. Make an HTTP GET request to get the [taxonomies:taxonomy](#) resource of the taxonomy that contains the configured categories.

Find the link [taxonomies:entries](#) and make an HTTP GET request to get a taxonomy:entries resource with the first page of entries in the form of an embedded array of taxonomies:entry resources. The resource uses paging to limit the message size. Example:

None

```
{
  "paging": {
    "limit": 25,
    "offset": 0,
    "elements": 2,
    "totalElements": 2
  },
  "_links": { ... },
  "_embedded": {
    "taxonomies:entry": [
      {
        "id": "category1",
        "labels": {
          "en": "category1"
        },
        "_links": { ... }
      },
      {
        "id": "category2",
        "labels": {
          "en": "category2"
        },
        "_links": { ... }
      }
    ]
  }
}
```

1. Repeat as long as a "next" link exists: make an HTTP GET request to the "next" link to get a taxonomy:entries resource with the next page of entries.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Categories/CTMS Get Categories of a System.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[taxonomies:taxonomy-by-taxonomyid](#)" link present inside the "resources", and store the variable globally as `selected_system_taxonomyByTaxonomyIdHref`.

GET 2_taxonomy by taxonomyid

- Hitting the above fetched `selected_system_taxonomyByTaxonomyIdHref`, to get the taxonomies for the given ID:"com.avid.workgroup.Property.Category".
- In the **Scripts** section: We fetch the "[taxonomies:entries](#)" link present inside the `_links`, and store it globally as `selected_system_taxonomyEntriesHref`, showing the list of taxonomy entries for the given system.

GET 3 taxonomy entries

- Hitting the above fetched `selected_system_taxonomyEntriesHref`, to get the taxonomy entries for the system.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Get categories of an asset in CTMS

Parameters:

- The MOB ID of the asset.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the masterclip and expand the URI template.
2. Add the query parameters
"embed=attributes&attributes=com.avid.workgroup.Property.Category".
3. Make an HTTP GET request to get the [aa:asset](#) resource with an embedded [aa:attributes](#) resource.

Find the attribute "com.avid.workgroup.Property.Category" in the "attributes" property of the embedded [aa:attributes](#) resource. The property "value" contains the ID and label of the category. Example:

None

```
{
  "base": {...},
  "common": {...},
  "_links": {...},
  "_embedded": {
    "aa:attributes": {
      "attributes": [
        {
          "name": "com.avid.workgroup.Property.Category",
          "value": [
            {
              "index": 0,
              "value": "category1"
            },
            {
              "index": 1,
              "value": "category2"
            }
          ],
          "type": "multi-value list",
          "taxonomyid": "com.avid.workgroup.Property.Category"
        }
      ]
    }
  }
}
```

```
    "_links": {...}
  }
}
```

You can also use the link [aa:asset-by-id](#)-bulk in step 2 to query multiple assets with a single bulk call. Avid recommends to keep the number of assets per request small (≤ 20).

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Categories/CTMS Get Categories of an Asset.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:update-asset-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_aaUpdateAssetByIdHref`.

◆ At this step, you need to insert the appropriate **master clip ID** in the environment variable named: `masterClipId`

GET 2_aa asset by id

- Hitting the above fetched `selected_system_aaUpdateAssetByIdHref`, to get the asset information for the given asset ID.
- In the **Params** section: We add the query parameters as (i) key as `embed`, value as `attributes`, and (ii) key as `attributes`, value as `com.avid.workgroup.Property.Category`.

The query parameter overall should look like:
?embed=attributes&attributes=com.avid.workgroup.Property.Category

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.
-

Get categories of a folder

Parameters:

- The ID of the folder, consisting of the path with leading and trailing slashes. Example: "/Projects/Example/".

Steps:

1. Query the information from the CTMS Registry.
2. Find the link [loc:item-by-id](#) for the Avid Production Management. The "href" is an URI template.
 1. Set the URI template variable "id" to the ID of the folder and expand the URI template.
 2. Add query parameter "attributes=com.avid.workgroup.Property.Category".
 3. Make an HTTP GET request to get the [loc:item](#) resource of the folder.
3. Find the attribute "com.avid.workgroup.Property.Category" in the property "attributes". The structure of the attribute is the same as above for assets.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Categories/CTMS Get Categories of a Folder.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources", and store

the variable globally as `selected_system_locItemByIdHref`.

◆ At this step, you need to insert the appropriate **folder ID** in the environment variable named: `folderIdInput`

GET 2_loc item by id

- Hitting the above fetched `selected_system_locItemByIdHref`, to get the folder structure information for the given folder ID.
- In the **Params** section: We add the query parameters as (i) key as `attributes`, value as `com.avid.workgroup.Property.Category`. The query parameter overall should look like: `?attributes=com.avid.workgroup.Property.Category`

Precautions/Recommendations:

- Please **select and put** the right folder IDs in the environment required for this example.

Documentation Title: Check in AAF

Summary:

The Interplay Web Services method `CheckInAAF` accepts a binary AAF file (Advanced Authoring Format) and checks-in the AAF to a given folder. If an asset with the MOB ID in the AAF already exists, the asset is updated, otherwise a new asset is created. The asset is added to the target folder if required.

Check-in an AAF in CTMS

Parameters:

- A byte array with the content of an AAF file.
- The ID of the target folder, consisting of the path with leading and trailing slashes.
Example: `"/Projects/Example/"`.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [loc:item-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the target folder and expand the URI template.
2. Make an HTTP GET request to get the [loc:item](#) resource of the folder.

Make an HTTP POST request to the pa:upload-file link.

1. Set header Content-Type: application/octet-stream.
2. Send the bytes of the AAF in the request body.
3. The response is a JSON with two properties: name is the file name of the uploaded file, downloadLink is a URI that can be used to download the file again:

None

```
{
  "name": "a4a43ca8-17c8-4983-a52e-56878c8d4f72.dat",
  "downloadLink":
  "https://.../a4a43ca8-17c8-4983-a52e-56878c8d4f72.dat"
}
```

Make an HTTP POST call to the pa:import-asset-command link of the [loc:item](#) resource from step 2 to start a long-running command that checks-in the AAF.

1. The request body is a JSON with a property "filename" that contains the value of the received "name" property from step 3:

None

```
{
  "filename": "a4a43ca8-17c8-4983-a52e-56878c8d4f72.dat"
}
```

2. The response is a command:command resource representing the command. Example response (abbreviated):

None

```
{
```

```

"version": "1.0",
"command": {
  "type": "aa:import-asset-command",
  "lifecycle": "pending"
},
"_links": {
  "self": {
    "href": "https://..."
  }
}
}

```

Poll the command status (with a small delay of, for example, 200 milliseconds between calls) by making an HTTP GET call to the "self" link until "lifecycle" in "command" is either "finished" or "error". The result depends on the final value of the "lifecycle" property:

1. "finished": The export process was completed successfully. The response contains an embedded [loc:item](#) resource with the reference to the checked-in asset, including an embedded loc:referenced-object resource with information about the asset. Example (abbreviated):

```

None
{
  "version": "1.0",
  "command": {
    "type": "pa:import-asset-command",
    "lifecycle": "finished"
  },
  "_embedded": {
    "loc:item": {
      "base": {
        "id":
"/Catalogs/IPWS/060a2b340101010101010f0013-000000-55dd943eda0748b
d-060e2b347f7f-2a80"
      },
      "common": {

```

```

        "name": "Some asset"
    },
    "_embedded": {
        "loc:referenced-object": {
            "base": {
                "id":
"060a2b340101010101010f0013-000000-55dd943eda0748bd-060e2b347f7f-
2a80",
                "type": "masterclip"
            },
            "common": {
                "name": "Some asset"
            }
        }
    }
}
}

```

2. "error": The call failed. The response contains a property "error" within the "command" property with error details. See command:command for details.

The pa:upload-file and pa:import-asset-command links will be part of the release version of Avid Production Management and will be documented on developer.avid.com at that point in time.

See [SetHeadframe via CTMS](#) if you want to set the headframe for the asset.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Check in AAF/CTMS Check in AAF.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: `avid.ctms.registry`

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
 - In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_locItemByIdHref`.
- ◆ At this step, you need to insert the appropriate **folder ID** in the environment variable named: `folderIdInput`

GET 2_loc item by id

- Hitting the above fetched `selected_system_locItemByIdHref`, to get the location/folder structure details of the selected folder ID.
- In the **Scripts** section:
 - We fetch the "pa:upload-file" link from the `_links` in the HAL output, and store it in `selected_system_paUploadFileHref` variable globally.
 - We also fetch "pa:import-asset-command" from the `_links`, and store it in `selected_system_paImportAssetCommandHref` variable globally. Both of these links will be used in the upcoming steps.

PUT 3 upload file

- In the **Headers** section : Put Content-Type as "application/octet-stream".
- In the **Body** section: Add your aaf file as a binary.
- From the Response, get the filename and keep it with you for the next step.

POST 4 import asset command

- In the **Body** section: add the filename you retrieved in the last step.
- In the **Scripts** section: We fetch the "self" link from "`_links`", that point to the long running command and we can poll the command status using this link.

GET 5 poll command status

- Here we poll the status of the long running command that shows if the thumbnail has been set for the asset or not.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Create Folder

Create a folder in CTMS

Summary:

The Interplay Web Services method `CreateFolder` creates a folder or nested path of folders. The method `CreateFolders` creates multiple folders or nested paths of folders. Optionally, the owner of the folder can be set.

In CTMS you can create only one direct sub folder to an existing folder with a single call. To create a path of folders, you must create them subfolder by subfolder. To create multiple folders, you must also create them one by one.

Parameter:

- The ID of an existing folder, consisting of the path with leading and trailing slashes.
Example: `"/Projects/Example/"`.
- The name of the sub folder to create.

Steps:

Use the cached CTMS Registry information or fetch it as shown in [Using the CTMS API](#).

Find the link [loc:item-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the folder and expand the URI template.
2. Make an HTTP GET request to get the [loc:item](#) resource of the folder.

Call HTTP POST on the [loc:create-folder](#) link with a JSON message body like this:

None

```
{
  "common": {
    "name": "Name of new folder"
  }
}
```

In CTMS, the caller of [loc:create-folder](#) is set as owner of the new folder.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Create Folder/CTMS Create Folder User Input.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_locItemByIdHref`.

◆ At this step, you need to insert the appropriate **folder ID** in the environment variable named: `folderIdInput`

GET 2_loc item by id

- Hitting the above fetched `selected_system_locItemByIdHref`, to get the folder information for the given folder ID.
- In the **Scripts** section: We fetch "[loc:create-folder](#)" link to fetch the link for creating a new folder in the desired location, and store it globally as `selected_system_createSubFolder`.

POST 3_create Folder Inside Desired Location

- Hitting the above `selected_system_createSubFolder`, to create a sub folder inside the given path.
- In the **Body** section: we provide the "common" value with the name of the new sub folder that we want to create.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Get Custom User Attributes

Summary:

The Interplay Web Services method GetCustomUserAttributes returns the registered user attributes for the Avid Production Management workgroup.

Get known attributes via CTMS

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link [datamodel:system-model](#) for the Avid Production Management and make an HTTP GET request to get the [datamodel:system-model](#) resource.

The property "attributes" lists the known attributes of the Avid Production Management:

- "common" contains information about the common attributes. They are returned in the "common" property of assets ([aa:asset](#) resource) and folders ([loc:item](#) resource).
- "custom" contains information about the raw user and system attributes of the Avid Production Management. They are returned as property "attributes" for folders, and as property "attributes" in an embedded [aa:attributes](#) resource for assets, when requested with query parameter "?attributes=*" (to get all attributes) or "?attributes=a,b,c" (to get a comma-separated list of given attributes).

User attributes in Avid Production Management are the ones with names that start with the prefix "com.avid.workgroup.Property.User".

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Custom User Attributes/CTMS Get Custom User Attributes.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: `avid.ctms.registry`

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[datamodel:system-model](#)" link present inside the "resources", and store the variable globally as `selected_system_dataModelSystemModelHref`.

GET 2 data model system model

- Hitting the above fetched `selected_system_dataModelSystemModelHref`, to get the custom user attributes in the output.
- In the **Params** section: You can set the parameter "attributes" as mentioned above in the summary tab.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Delete Assets (Bulk)

Summary:

The Interplay Web Services method `DeleteAssets` is used to delete assets, associated media, and/or empty folders. It accepts parameters that define what exactly to delete:

- `DeleteMetadata`: If true, the metadata of the asset is deleted from the folder.
- `DeleteMedia`: If true, the media files referenced by the asset are deleted.
- If `DeleteMedia` is true:
 - `Resolutions`: Defines the resolutions of the media files to delete. If not given, media files of all resolutions are deleted.
 - `FileType`: Defines the type of the media files to delete (AMA, NATIVE or ALL). The method also allows to only simulate the deletion. In this case, the result is a list of the assets and media files that would be deleted.

Deleting empty folders in CTMS

CTMS currently doesn't allow deleting empty folders in Avid Production Management.

Deleting assets in CTMS

Parameter:

- A list of asset references in folders. An asset reference consists of the path in the folder structure, followed by the MOB ID of the asset. Example:
"/Catalogs/IPWS/060a2b340101010101010f0013-000000-6dda7dad08c24cfd-b581d7f05b5b-51fb".
- The CTMS deletion mode:

Deletion mode	Description
all	Delete items in a folder and the referenced assets (if the item is the last reference to the asset), including the media files. Equivalent to DeleteMetadata=true, DeleteMedia=true in IPWS.
metadata	Delete items in a folder and the referenced assets (if the item is the last reference to the asset). This is the default if "mode" is not given in the deletion request. Equivalent to DeleteMetadata=true, DeleteMedia=false in IPWS.
media	Delete all (online) media files of the referenced assets. Items and referenced assets are not deleted, but will become offline. Equivalent to DeleteMetadata=false, DeleteMedia=true in IPWS.

CTMS doesn't support selecting only specific resolutions or file types. The deletion modes "media" and "all" will delete all media files (if the last reference to the asset is deleted).

CTMS doesn't support simulating the deletion.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [loc:locations](#) for the Avid Production Management and make an HTTP GET request.

Find the link [loc:delete-item-by-id-bulk-command](#).

- The request body is a JSON array with the IDs of the asset references to delete.
Example

None

```
[  
  "/Catalogs/IPWS/060a2b3401010101010f0013-000000-6dda7dad08c24cf  
d-b581d7f05b5b-51fb",  
  "/Catalogs/IPWS/060a2b3401010101010f0013-000000-070812007d07001  
8-060e2b347f7f-d080"  
]
```

- Set the URI query parameter "mode" to the desired deletion mode.
- Make an HTTP POST request to start a long-running command that deletes the assets. The response is a command:command resource that represents the command. Example (abbreviated):

None

```
{  
  "version": "1.0",  
  "command": {  
    "type": "loc:delete-item-by-id-bulk-command",  
    "lifecycle": "pending"  
  },  
  "_links": {  
    "self": {  
      "href": "https://..."  
    }  
  }  
}
```

1. Poll the command status (with a small delay of, for example, 200 milliseconds between calls) by making an HTTP GET call to the "self" link until "lifecycle" in "command" is either "finished" or "error".
2. The result depends on the final value of the "lifecycle" property:

- “finished”: The deletion process was finished. It's possible that only a subset of the assets was really deleted. See below for evaluating the result.
- “error”: The entire deletion command failed. The response contains a property “error” within the “command” property with error details. See command:command for details.

If the final value of “lifecycle” is “finished”, the property “result” in “payload” contains an array with the deletion result for each requested ID, in the same order as in the request body:

```
None
...
{
  "version": "1.0",
  "command": {
    "type": "loc:delete-item-by-id-bulk-command",
    "lifecycle": "finished"
  },
  "payload": {
    "command-parameters": {
      "ids": [

"/Catalogs/IPWS/060a2b340101010101010f0013-000000-6dda7dad08c24cf
d-b581d7f05b5b-51fb",

"/Catalogs/IPWS/060a2b340101010101010f0013-000000-070812007d07001
8-060e2b347f7f-d080"
      ]
    },
    "result": [
      {
        "success": true,
        "data":
"/Catalogs/IPWS/060a2b340101010101010f0013-000000-6dda7dad08c24cf
d-b581d7f05b5b-51fb"
      },
      {
        "success": false,
        "httpStatus": 400,
```

```

        "errorCode": "avid.mam.assets.access/BAD_ARGUMENT",
        "errorMessage": "Invalid item ID"
    }
]
}
}
...

```

Each result item has the following properties:

- success: true if the deletion was successful.
- Only if success is false:
 - httpStatus: the HTTP status code that a non-bulk call would return.
 - errorCode: the CTMS error code.
 - errorMessage: the error message.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Delete Asset/CTMS Delete bulk Assets.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:locations](#)" link present inside the "resources", and store the variable globally as selected_system_locLocationsHref.

GET 2_loc location href

- Hitting the above fetched selected_system_locLocationsHref, to get the location or folder information of the system we are in.
- In the **Scripts** section: From the response, we fetch the "[loc:delete-item-by-id-bulk-command](#)", which is used to delete the items in bulk using this

one command, and store it globally as
selected_system_locDeleteItemByldBulkCommandHref.

POST 3 delete assets in bulk

- Hitting the above selected_system_locDeleteItemByldBulkCommandHref URL, we run the deletion process.
- In the **Body** section: **You as a user have to provide the list of assets or asset paths inside the array that you want to delete in your system.**
- In the **Scripts** section: We fetch the "self" link from "_links", that point to the long running command and we can poll the command status using this link.

◆ At this step, you need to insert the appropriate **asset paths** in the request body array

GET 4 poll command status (pending/finished)

- Here we poll the status of the long running command that shows if the assets have been deleted or not.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.
- It is suggested to have a separate folder with assets you want to delete and only perform this action with caution on those selected assets. Once deleted, assets can not be retrieved back.

Documentation Title: Duplicate an Asset

Summary:

The Interplay Web Services method Duplicate is used to duplicate an asset. The duplicated asset will get a new MOB ID. The method accepts the following parameters:

- InterplayAssetURI: The URI of the asset to duplicate.
- InterplayFolderURI: The URI of the target folder for the duplicated asset.
- Attributes: attributes to set for the duplicated asset.

Duplicating an asset in CTMS

Parameter:

- The MOB ID of the asset to duplicate.
- The ID of the target folder.
- Optional: the name of the duplicated asset. If not given, the duplicated asset will get a default name derived from the name of the original asset.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the asset to duplicate and expand the URI template.
2. Make an HTTP GET request to get the [aa:asset](#) resource for the asset.

Make an HTTP POST request to the link aa:duplicate-asset. The request body is a JSON object with the name of the duplicated clip as common attribute "name" and the ID of the target folder in property "location":

None

```
{
  "common": {
    "name": "My duplicated asset"
  },
  "location": "/Catalogs/IPWS/"
}
```

The response is a JSON object with two properties:

None

```
{
  "aa:asset": {
    "success": true,
```

```
"data": {
  "base": {
    "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
    "systemType": "avid-pmplus",
    "id":
"/060a2b34010101010101010f0013-000000-0b1e0e003a6d00fd-060e2b347f7f-d080",
    "type": "masterclip"
  },
  "common": {
    "name": "Duplicated asset",
    ...
  },
  "_links": { ... }
},
"loc:item": {
  "success": true,
  "data": {
    "base": {
      "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
      "systemType": "avid-pmplus",
      "id":
"/Catalogs/IPWS/060a2b34010101010101010f0013-000000-0b1e0e003a6d00fd-060e2b347f7f-d080",
      "type": "folder-item"
    },
    "common": {
      "name": "Duplicated asset",
      ...
    },
    "_links": { ... },
    "_embedded": {
      "loc:referenced-object": { ... }
    }
  }
}
```

```
}  
}
```

The property [aa:asset](#) contains the information about the created asset. The format is the same as in the response for bulk calls with the sub properties:

1. success: true if the asset was created successfully, false if not.
2. data: in case of success, the property contains the [aa:asset](#) resource of the created subclip, including the MOB ID in the property "id" in "base".
3. httpStatus, errorCode and errorMessage: in case of an error.

The property [loc:item](#) contains information about the created asset link in the target folder. The format is the same as above, only with the [loc:item](#) resource of the created subclip in the data property in case of success.

You can use the link aa:update-asset in the returned [aa:asset](#) resource to set common attributes and user attributes of the subclip. See "SetAttributes via CTMS" for details.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Duplicate Asset/CTMS Duplicate an Asset.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as selected_system_aaAssetByIdHref.

◆ At this step, you need to insert the appropriate **master clip ID** in the environment variable named: masterClipId

GET 2_aa asset by id

- Hitting the above fetched selected_system_aaAssetByldHref, to get the asset information by id.
- In the **Scripts** section: From the response, we fetch the "aa:duplicate-asset", which is used to create a duplicate for the selected asset, and store it globally as selected_system_aaDuplicateAsset.

POST 3 duplicating an asset

- Hitting the above selected_system_aaDuplicateAsset URL, we create the duplicate for the selected asset.
- In the **Body** section: We provide the duplicated asset details, like the name and location where we need to store this duplicated item finally.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Get Children

Getting the folder content in CTMS

Summary:

The Interplay Web Services method GetChildren returns the content of a folder. There are a number of parameters for this function:

- IncludeMOBs: defines if Avid assets are returned or not. Default: true.
- IncludeFolders: defines if folders are returned or not. Default: true.
- IncludeRefAssets: defines if referenced assets are returned or not. Default: false. If you store, for example, a sequence in a folder, then the masterclips that are used in the sequence are also added silently as "referenced assets" to the folder.
- ReturnAttributes: defines attributes to return for each item in the folder. If not given, a default set of attributes are returned.
- Filter: regular expression filter on the names of items.

Parameter:

- The ID of the folder, consisting of the path with leading and trailing slashes. Example: `"/Projects/Example/"`.
- Optional: a list of attributes to get.
- Optional: a filter for the items to return.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link [loc:item-by-id](#) for the Avid Production Management. The "href" is an RFC-6570 URI template.
 1. Set the URI template variable "id" to the ID of the folder and expand the URI template.
 2. Add the query parameter "attributes=*" to get all attributes, or "attributes=a,b,c" to get a specific list of attributes.
 3. Add the query parameter "filter" to control which items to return. The value is a comma-separated list of the following values:
 - item-type-folder: return folders.
 - item-type-asset: return assets.
 - item-type-referenced-asset: return referenced assets. If filter is not given, the method returns folders and asset, similar to `"filter=item-type-folder,item-type-asset"`.
 4. Make an HTTP GET call on the resulting URI. The result is the `loc:item` resource for the requested folder with an embedded `loc:collection` resource having an embedded list with the first page of `loc:item` resources for the items in the folder. See below for an example.
3. The [loc:collection](#) resource uses paging. To get all items, find the "next" link in it. As long as the "next" link exists, add the same query parameters as above and use HTTP GET to get the next page.

CTMS doesn't support a regular expression as filter for items to return. You must apply the filter on the client side.

Structure of the `loc:item` response

The result of step 2.4 above has the following structure:

- base, common, and attributes: information for the requested folder.
- `_embedded`
 - `loc:collection`: the collection of items in the folder
 - paging: information about the items that are returned by this call, and the total number of items in the folder
 - `_links`: HAL links of the collection
 - next: link to the next page of results. Only present if there are more pages.
 - `_embedded`:

- `loc:item`: embedded array of `loc:item` resources for the items in the collection. This can be either sub folders or assets that are in the requested folder.
 - `base`: ID of the item in the folder
 - `common`: common attributes like display name for the item.
 - `attributes` (only for sub folders): user and system attributes of the sub folder
 - `_links`: links of the `loc:item`. If it contains a `loc:collection` link, the item is a folder.
 - `_embedded` (only for assets)
 - `loc:referenced-object`: the `aa:asset` resource of the asset in the folder
 - `base`: ID of the asset
 - `common`: common attributes like display name for the asset
 - `_embedded`
 - `aa:attributes`: user and system attributes for an asset.

When following the "next" link of the collection, the structure is the same as the one for the embedded `loc:collection` resource above.

Abbreviated example for the result with a first page with two items, an asset and a sub folder:

```
None
{
  "base": {...},
  "common": {...},
  "_links": {...},
  "_embedded": {
    "loc:collection": {
      "paging": {
        "limit": 2,
        "offset": 0,
        "elements": 2,
        "totalElements": 7
      },
      "_links": {
        "next": {
          "href": "https://..."
        }
      }
    }
  }
}
```

```
},
...
},
"_embedded": {
  "loc:item": [
    {
      "base": {
        "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
        "systemType": "avid-pmplus",
        "id":
        "/Catalogs/IPWS/060a2b340101010101010f0013-000000-00c5dfe08ab0de8
        0-060e2b347f7f-2a80",
        "type": "folder-item"
      },
      "common": {
        "creator": "svceditor",
        "created": "2017-03-16T16:01:14Z",
        "name": "Example asset"
      },
      "_links": {...},
      "_embedded": {
        "loc:referenced-object": {
          "base": {
            "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
            "systemType": "avid-pmplus",
            "id":
            "060a2b340101010101010f0013-000000-00c5dfe08ab0de80-060e2b347f7f-
            2a80",
            "type": "masterclip"
          },
          "common": {
            "creator": "svceditor",
            "created": "2017-03-16T16:01:14Z",
            "name": "Example asset"
          },
          "_links": {...},
```

```
"_embedded": {
  "aa:attributes": {
    "attributes": [
      {
        "name": "com.avid.workgroup.Property.System.Format",
        "value": "1080i/50",
        "type": "string"
      }
    ],
    "_links": {...}
  }
}
},
{
  "base": {
    "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
    "systemType": "avid-pmplus",
    "id": "/Catalogs/IPWS/A sub folder/",
    "type": "folder"
  },
  "common": {
    "path": "/Catalogs/IPWS/A sub folder/",
    "creator": "svcadmin",
    "created": "2024-04-16T13:01:42.204Z",
    "name": "A sub folder",
    "modified": "2024-04-16T13:01:42.204Z"
  },
  "attributes": [
    {
      "name": "Created By",
      "value": "svcadmin",
      "type": "string"
    }
  ],
}
```

```

    "_links": {...}
  }
]
}
}
}
}
}
}

```

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Get Children\ Avid Production Management CTMS-Get Children.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call get logged in the system. View the project-level ReadMe to understand how it works.)

GET 1_avid.ctms.registry -> getServiceRoots -> set locItemByld from global folderID

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section, we check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources" for "avid-pmplus" systemType, and store the variable globally as **selected_system_locItemByldHref**. Here we also replace "id" parameter in the url with user input "folder_path".

◆ At this step, you need to insert the appropriate **folder_path** in the environment variable.

Get 2.1_Get Loc:item by ID(w/o filter)

- Hitting the above fetched **selected_system_locItemByldHref**, to get ["loc:item"] resource for a folder. The result is the loc:item resource for the requested folder with an

embedded loc:collection resource having an embedded list with the first page of loc:item resources for the items in the folder.

Get 2.2_Get Loc:items by ID(with attr)

- Hitting the above **selected_system_locItemByIdHref** URL with **attributes** as a query parameter.
- Here we pass **attributes** as a query parameter. e.g, "attributes = Name,SMOT" The result is the loc:item resource for the requested folder with an embedded loc:collection resource with selected list of attributes in the "attributes" array.

Get 2.3_Get Loc:items by ID(with filter)

- Hitting the above "selected_system_locItemByIdHref" URL with **filter** as a query parameter.
- Here we pass **filter** as a query parameter to control which items to return. Below are values we can pass in a filter. ** filter = item-type-folder (return folders) ** ** filter = item-type-asset (return assets) ** ** filter = item-type-referenced-asset (return referenced assets) **

The result is the filtered loc:item resource for the requested folder with an embedded loc:collection resource having an embedded list with the first page of loc:item resources for the items(children) in the folder.

Precautions/Recommendations:

- Please select and put the right **folder_path** in the environment required for this example.

Documentation Title: Get File Details

Get file details for a masterclip in CTMS

Summary:

The Interplay Web Services method GetFileDetails returns information about the media files used by a masterclip.

Parameter:

- The MOB ID of the masterclip.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link aa:asset-by-id for the Avid Production Management. The "href" is a URI template.
 1. Set the URI template variable "id" to the ID of the asset and expand the URI template.
 2. Make an HTTP GET request to get the aa:asset resource for the asset
3. Make an HTTP GET request to the pa:asset-medias link to get the information about the files used by the masterclip in the property "media-items".

Example response (abbreviated):

None

```
{
  "media-items": [
    {
      "name":
"060a2b340101010101010f0013-000000-9a855fb4b5e900a5-5d76f1bbe9eb-
b3a6",
      "track": "A1",
      "online": "true",
      "filePath": "//somehost/.../2013060413a018c27d65.mxf",
      "type": "filemob",
      "essenceType": "NATIVE",
      "createdBy": "Administrator",
      "format": "PCM",
      "size": "2816"
    },
    {
      "name":
"060a2b340101010101010f0013-000000-9b855fb4b5e900a5-eaaddb3c870c-
3e99",
      "track": "A2",
      "online": "true",
      "filePath": "//somehost/.../2013060413a028c26f7f.mxf",
      "type": "filemob",
      "essenceType": "NATIVE",
      "createdBy": "Administrator",
```

```

    "format": "PCM",
    "size": "2816"
  },
  {
    "name":
"060a2b340101010101010f0013-000000-9a855fb4b5e900a5-8be8d6ae5290-
49f1",
    "track": "V1",
    "online": "true",
    "filePath": "//somehost/.../2013060413v018c21529.mxf",
    "type": "filemob",
    "essenceType": "NATIVE",
    "createdBy": "Administrator",
    "format": "XDCAM-HD 50mbps 1080p 25",
    "size": "141824"
  }
]
}

```

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Get File Details\CTMS Get File Details.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to get logged in the system. View the project-level ReadMe to understand how it works.)

GET 1_avid.ctms.registry -> getServiceRoots -> set assetById from global var master_clip_id

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id]"(<https://developer.avid.com/ctms/api/aa/linkrels/asset-by-id.html>)" link present inside the "resources" for 'selected_system' systemType, and store the variable

globally as "selected_system_assetByIdHref". Here we also replace "id" parameter in the url with the ID of the masterclip.

◆ At this step, you need to insert the appropriate **master_clip_id** in the environment variable.

Get 2_asset-by-id -> asset-medias

- Hitting the above fetched 'selected_system_assetByIdHref', to get asset medias link relation for the given asset.
- In the **Scripts** section: From the response, we fetch the "[pa:asset-medias]" href, which will be used to get the information about the files used by the masterclip in the property "media-items", and store it globally as "selected_system_assetmedias".

GET 3_asset-medias -> media-items

- Hitting the above "selected_system_assetmedias" URL to get the asset media information. The response contains information about the files used by the masterclip in the property "media-items".

Precautions/Recommendations:

- Please **select and put** the right **master_clip_id** in the environment required for this example.

Documentation Title: Headframe/Thumbnails

Set HeadFrame/Thumbnails

Summary:

The Interplay Web Services method SetHeadframe sets the headframe (or "thumbnail") image for an asset.

Parameter:

- The MOB ID of the asset.
- The binary image.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.
 1. Set the URI template variable "id" to the ID of the asset and expand the URI template.
 2. Make an HTTP GET request to get the [aa:asset](#) resource for the asset.
3. Find the [aa:set-thumb-command](#) link.
 1. Send the byte array of the image in the request body.
 2. Set the HTTP header "Content-Type: image/jpeg"
 3. Make an HTTP POST call to the link to start a long-running command that sets the thumbnail. The response is a command:command resource representing the long-running command. Example response (abbreviated):

```
None
{
  "version": "1.0",
  "command": {
    "lifecycle": "pending"
  },
  "_links": {
    "self": {
      "href": "https://..."
    }
  }
}
```

4. Poll the command status (with a small delay of, for example, 200 milliseconds between calls) by making an HTTP GET call to the "self" link until "lifecycle" in "command" is either "finished" or "error".
5. The result depends on the final value of the "lifecycle" property:
 1. "finished": The thumbnail has been set successfully.
 2. "error": The call failed. The response contains a property "error" within the "command" property with error details. See command:command for details.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "HeadFrames or Thumbnails/CTMS Set Headframe.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_aaAssetByldHref`.

◆ At this step, you need to insert the appropriate **master clip ID** in the environment variable named: `masterClipId`

GET 2_aa asset by id

- Hitting the above fetched `selected_system_aaAssetByldHref`, to get the asset by id.
- In the **Scripts** section: From the response, we fetch the "[aa:set-thumb-command](#)", which is used to set the thumbnail information for the given asset, and store it globally as `selected_system_aaSetThumbCommandHref`.

POST 3 set a thumbnail

- Hitting the above `selected_system_aaSetThumbCommandHref` URL, to set the thumbnail information.
- In the **Body** section: Select 'binary' as the input and select the thumbnail image of your choice. You can choose the file present in the /extras/ folder as well.
- The response starts a long running command that sets the thumbnail, with lifecycle initially as pending, and once completed, turns to finished.
- In the **Scripts** section: We fetch the "self" link from "_links", that point to the long running command and we can poll the command status using this link.

GET 4 poll command status (pending/finished)

- Here we poll the status of the long running command that shows if the thumbnail has been set for the asset or not.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

- Please update the image or choose a new one in case, you get an error saying "INTERNAL SERVER ERROR".
 - Feel free to play around with the image you can use as the thumbnail at step 3.
-

Get HeadFrame/Thumbnails

Summary:

The Interplay Web Services method GetHeadframe returns a headframe (or "thumbnail") image for an asset.

Parameters

- The MOB ID of the asset.

Steps

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the asset and expand the URI template.
2. Add the query parameter "embed=thumb".
3. Optional: add the query parameters "thumbnailWidth" and "thumbnailHeight" to request the image in a specific resolution. See below for details.
4. Make an HTTP GET request to get the [aa:asset](#) resource for the asset with an embedded [aa:thumb](#) resource. The property "thumbnail" has either the URL of the thumbnail image as value, or null to indicate that the asset has no thumbnail. Example:

None

```
{
  "base": { ... },
  "common": { ... },
  "_embedded": {

    "[aa:thumb](https://developer.avid.com/ctms/api/aa/linkrels/thumb.html)": {
      "thumbnail": "https://...",
      "_links": {
```

```
}  
}  
}  
}
```

1. If "thumbnail" is not null, call HTTP GET on the URL to get the image.

CTMS returns the image for a Avid Production Management asset by default as a JPEG image scaled to a configured default resolution. You can use the query parameters "**thumbnailWidth**" and "**thumbnailHeight**" to request the image in a specific size. CTMS will return an image of at most the given size but may also return a smaller image.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "HeadFrames or Thumbnails/CTMS Get Headframe.postman_collection.json", and follow the steps as given below.

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_aaAssetByldHref`.

◆ At this step, you need to insert the appropriate **master clip ID** in the environment variable named: `masterClipId`

GET 2_aa asset by id

- Hitting the above fetched `selected_system_aaAssetByIdHref`, to get the asset by id.
- In the **Scripts** section: From the response, we fetch the "[aa:thumb](#)", which is used to get the thumbnail information for the given asset, and store it globally as `selected_system_thumbnailHref`.

GET 3 get a thumbnail

- Hitting this URL will load the thumbnail image for the Avid Production Management asset.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Get Latest AAF

Summary:

The Interplay Web Services method `GetLatest` returns the latest version of the AAF of an asset.

Parameter:

- The MOB ID of the asset.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.
 1. Set the URI template variable "id" to the ID of the asset and expand the URI template.
 2. Make an HTTP GET request to get the [aa:asset](#) resource for the asset
3. Make an HTTP GET request to the `pa:export-asset-command` link to start a long-running command that exports the AAF. The response is a `command:command` resource representing the command. Example response (abbreviated):

None

```
{
  "version": "1.0",
```

```

    "command": {
      "type": "aa:export-asset-command",
      "lifecycle": "pending"
    },
    "_links": {
      "self": {
        "href": "https://..."
      }
    }
  }
}

```

Poll the command status (with a small delay of, for example, 200 milliseconds between calls) by making an HTTP GET call to the "self" link until "lifecycle" in "command" is either "finished" or "error".

The result depends on the final value of the "lifecycle" property:

1. "finished": The export process was completed successfully. In this case, the response contains a pa:download-file link:

None

```

"version": "1.0",
"command": {
  "type": "aa:export-asset-command",
  "progress": 100,
  "lifecycle": "finished"
},
"_links": {
  "self": {
    "href": "https://..."
  },
  "pa:download-file": {
    "href": "http://..."
  }
}

```

}

2. "error": The call failed. The response contains a property "error" within the "command" property with error details. See command:command for details.
5. If the command was successful, make an HTTP GET call on the pa:download-file link to download the AAF.

The aa:export-asset-command and pa:download-file links will be part of the release version of Avid Production Management and will be documented on developer.avid.com at that point in time.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Latest AAF/CTMS Get Latest AAF.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as selected_system_aaAssetByldHref.

◆ At this step, you need to insert the appropriate **asset ID** in the environment variable named: assetMobld

GET 2_aa asset by id

- Hitting the above fetched selected_system_aaAssetByldHref, to get the asset by id.

- In the **Scripts** section: From the response, we fetch the "aa:export-asset-command", which is used to get the export link for the given asset, and store it globally as selected_system_aaExportAssetCommandHref.

POST 3 export AAFs

- Hitting the above selected_system_aaExportAssetCommandHref URL, to export the latest AAF file.
- The response starts a long running command that exports the file, with lifecycle initially as pending, and once completed, turns to finished.
- In the **Scripts** section: We fetch the "self" link from "_links", that point to the long running command and we can poll the command status using this link.

GET 4 poll command status (pending/finished)

- Here we poll the status of the long running command that shows if the thumbnail has been set for the asset or not.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Move Item

Summary:

The Interplay Web Services method Move moves an asset from a source folder to a target folder, or an entire folder to a target folder. If a sequence is moved, the referenced assets are also moved (or copied if they are referenced by another sequence in the source folder) to the target folder.

The method allows overriding reservations if needed.

Move an asset or folder to a target folder in CTMS

Parameter:

- The ID of the item to move:
 - For an asset: the path to the asset with a leading "/". The last path element is the MOB ID. Example:

"/Catalogs/IPWS/060a2b340101010101010f0013-000000-070812007d070018-060e2b347f7f-d080"

- For a folder: the path with leading and trailing slashes. Example: "/Catalogs/IPWS/".
- The ID of the target folder, consisting of the path with leading and trailing slashes. Example: "/Projects/Example/".

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [loc:item-by-id](#) for the Avid Production Management. The "href" is an URI template.

- Set the URI template variable "id" to the ID of the target folder and expand the URI template.
- Make an HTTP GET request to get the [loc:item](#) resource of the folder.

Find the [loc:move-item](#) link.

- If you want to override reservations, set the query parameter "override-reservations" to "all".
- Make an HTTP POST call with a JSON object in the request body with a property "base" with the sub properties:
 - "systemType": the Avid Production Management type "avid-pmplus".
 - "systemID": the ID of the Avid Production Management.
 - "type": "folder" for a folder, and "folder-item" for an asset.
 - "id": the path to the asset or folder.

Example:

None

```
{
  "base": {
    "systemType": "avid-pmplus",
    "systemID": "DD481E2E-260B-4684-9D25-44C774E3F782",
    "type": "folder-item",
    "id":
"/Catalogs/IPWS/060a2b340101010101010f0013-000000-070812007d070018-060e2b347f7f-d080"
  }
}
```

}

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Move Item/CTMS Move Item.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_locItemByIdHref`.

GET 2_loc item by id

- Hitting the above fetched `selected_system_locItemByIdHref`, to get the loc item by id which represents the location of the item.
- In the **Scripts** section: From the response, we fetch the "[loc:move-item](#)", which is used to get the link to move an item, and store it globally as `selected_system_locMoveItem`.

POST 3_create Folder Inside Desired Location

- Hitting the above `selected_system_locMoveItem` URL, to move the item to the desired location.
- In the **Body** section: We put the "base" information of the item that we want to move, as defined above in the summary tab.

◆ At this step, you need to insert the appropriate **systemType**, **systemID**, **type** and **id** in the "base" section of the request body that we are sending.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.
- Please make sure to use the correct item id, type etc., in the request body at step 3.

Documentation Title: References To An Asset (FindLinks in IPWS)

Summary:

The Interplay Web Services method FindLinks returns all references to a given asset in the entire folder structure.

Get all references to an asset in CTMS

Parameter:

- The MOB ID of the asset.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the asset and expand the URI template.
2. Make an HTTP GET request to get the [aa:asset](#) resource for the asset

Make an HTTP GET request to the [loc:referencing-items](#) link to get a [loc:referencing-items](#) resource with an embedded array of [loc:item](#) resources with all references to the asset in the entire folder hierarchy. The IDs of the references can be found in the property "id" in "base".
Example response (abbreviated):

None

```
{
  "_embedded": {
    "loc:item": [
```

```

{
  "base": {
    "id":
"/Catalogs/IPWS/Example/060a2b340101010101010f0013-000000-6dda7da
d08c24cfd-b581d7f05b5b-51fb",
    "type": "folder-item",
    "systemType": "avid-pmplus",
    "systemID": "DD481E2E-260B-4684-9D25-44C774E3F782"
  }
},
{
  "base": {
    "id":
"/Catalogs/JaneSmith/060a2b340101010101010f0013-000000-6dda7dad08
c24cfd-b581d7f05b5b-51fb",
    "type": "folder-item",
    "systemType": "avid-pmplus",
    "systemID": "DD481E2E-260B-4684-9D25-44C774E3F782"
  }
}
]
}
}

```

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "References to an Asset (FindLinks)/CTMS Find Links or References to an Asset.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
 - In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_aaAssetByldHref`.
- ◆ At this step, you need to insert the appropriate **asset ID** in the environment variable called, `assetMobld`.

GET 2_aa asset by id

- Hitting the above fetched `selected_system_aaAssetByldHref`, to get the asset by id in the system.
- In the **Scripts** section: From the response, we fetch the "[loc:referencing-items](#)", which is used to get the link to get all the references of an asset or links to that asset, and store it globally as `selected_system_locReferencingItemsHref`.

POST 3 get references/links to an asset

- Hitting the above `selected_system_locReferencingItemsHref` URL, to fetch all the associated links of that asset.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Relatives of an Asset

Summary:

The Interplay Web Services method `FindRelatives` returns assets that are related to a given asset. For example, the relatives of a masterclip are the sequences that use the masterclip, and the subclips that have been created from it. The response also contains attributes for each of the related assets.

Find related assets in CTMS

Parameter:

- The MOB ID of the asset.
- Optional: a list of attributes to get for each related asset.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the asset and expand the URI template.
2. Make an HTTP GET request to get the [aa:asset](#) resource for the asset.

Call HTTP GET on the relations:relations link. The response is a relations:relations resource containing the first page of the list of related assets.

Call HTTP GET on the "next" link to get the next page of related assets as long as the "next" link is present. Each page contains the paging information and an array of related entities (here, for example, with a page size of 1):

None

```
{
  "paging": {
    "limit": 1,
    "offset": 0,
    "elements": 1,
    "totalElements": 2
  },
  "_links": {
    "self": {
      "href": "https://..."
    },
    "first": {
      "href": "https://..."
    },
    "next": {
      "href": "https://..."
    },
    "relations:relation": [
      {
```

```

        "href": "https://..."
    }
]
},
"_embedded": {
    "relations:relation": [
        {
            "relationType": "masterclip.forward",
            "relatedEntity": {
                "id":
"060a2b340101010101010f0013-000000-00003d0f311a40e5-060e2b347f7f-
2a80",
                "kind": "aa",
                "type": "masterclip"
            },
            "labels": {
                "en": "Master Clip Relatives"
            },
            "_links": {
                "self": {
                    "href": "https://..."
                },
                "relations:related-entity": {
                    "href": "https://..."
                },
                "relations:relation-type": {
                    "href": "https://..."
                }
            }
        }
    ]
}
}

```

1. The properties "relatedEntity.id" and "relatedEntity.type" of the embedded relations:relation resources from all pages contain the IDs and types of the related assets.

If you need attributes for the related assets:

1. Find the link [aa:asset-by-id](#)-bulk in the CTMS Registry for the system ID of the Avid Production Management.
2. In batches of about 20 asset IDs: Use HTTP POST on the "href" of the [aa:asset-by-id](#)-bulk link with an array of asset IDs as body:

None

[

"060a2b340101010101010f0013-000000-00003d0f311a40e5-060e2b347f7f-2a80"

]

3. This returns a bulk response with an array in the same order as in the request body. For each requested asset, you get back either "success": false and error information, or "success": true and an [aa:asset](#) resource with common attributes and status values in the "data" property. See Bulk Calls for details.
4. If you want additional attributes, use the URL query parameters "?embed=attributes&attributes=X,Y,Z" to get an embedded [aa:attributes](#) resource with the additional attributes X, Y, and Z. Without the "attributes" query parameter, you get back all attributes.

Evaluate the attributes in the same way as for the method GetAttributes.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Relatives of Asset/CTMS Find Relatives or related assets.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
 - In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_aaAssetByldHref`.
- ◆ At this step, you need to insert the appropriate **asset ID** in the environment variable named: `assetMobld`

GET 2_aa asset by id

- Hitting the above fetched `selected_system_aaAssetByldHref`, to get the asset information by id.
- In the **Scripts** section: From the response, we fetch the "relations:relations", which is used to fetch the related assets to the given asset, and store it globally as `selected_system_relationsRelationsHref`.

GET 3 get related assets

- Hitting the above `selected_system_relationsRelationsHref` URL, to fetch all the relations to the given asset.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Rename

Rename folder or an asset in CTMS

Summary:

The Interplay Web Services method Rename changes the name of an asset or a folder.

Rename Folder

Parameter:

- The ID of the folder, consisting of the path with leading and trailing slashes. Example: `"/Projects/Example/"`.
- The new name.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link `loc:item-by-id` for the Avid Production Management. The `"href"` is an RFC-6570 URI template.
 1. Set the URI template variable `"id"` to the ID of the folder and expand the URI template.
 2. Make an HTTP GET request to get the `loc:item` resource of the folder.
3. Find the link `loc:update-item`.
 1. Make an HTTP PATCH request to the link with a `loc:item` resource that sets the common attribute `"name"` as request body.

Example:

```
None
{
  "common": {
    "name": "New name"
  }
}
```

Rename Asset

Parameter:

- The MOB ID of the asset
- The new name.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link `aa:update-asset-by-id` for the Avid Production Management. The `"href"` is an RFC-6570 URI template.
 1. Set the URI template variable `"id"` to the ID of the masterclip and expand the URI template.
 2. Make an HTTP PATCH request with an `aa:asset` resource. Set the common attribute `"name"` to the new name. Example:

Example:

None

```
{
  "common": {
    "name": "New name"
  }
}
```

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Rename\CTMS Rename Folder-Asset.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call get logged in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
 - In the **Scripts** section: a. We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources" for "selected_system" systemType, and store the variable globally as "selected_system_locItemByIdHref". Here we also replace "id" parameter in the url with user input "folder_path". b. We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:update-asset-by-id]"(<https://developer.avid.com/ctms/api/aa/linkrels/update-asset-by-id.html>)" link present inside the "resources" for "selected_system" systemType, and store the variable globally as "selected_system_updateAssetByIdHref". Here we also replace "id" parameter in the url with the ID of the masterclip.
- ◆ At this step, you need to insert the appropriate **folder_path** and **asset_id** in the environment variable.

Get 2_loc:item-by-id -> update-item

- Hitting the above fetched 'selected_system_locItemByIdHref', to get ["loc:update-item"] resource for a folder.

- In the **Scripts** section: From the response, we fetch the "[\"loc:update-item\"]", which will be used to rename a given folder, and store it globally as "selected_system_updateItemByIdHref".

PATCH 3_Rename Folder

- Hitting the above "selected_system_updateItemByIdHref" URL, to update the name of a given input folder.
- In the **Body** section: Add request body with the common attribute "name" field to add new name for the folder to be renamed. The response contains a modified loc:item resource with a new ID in the sub property "id" of property "base".

PATCH 4_Rename Asset

- Hitting the above "selected_system_updateAssetByIdHref" URL fetched in step 1, to update the name of an asset for a given assetId.
- In the **Body** section: Add request body with the common attribute "name" field to add new name for an asset to be renamed. The response contains a modified loc:item resource with a new name in the sub property "name" of property "common".

Precautions/Recommendations:

- Please select and put the right **folder_path** and **asset_id** in the environment required for this example.

Documentation Title: Reservations

Add Reservations in CTMS

Summary:

The Interplay Web Services method AddReservation adds a new reservation to a given list of folders. The reservation can be created for an unlimited time or it can have an expiration time (given in seconds).

Parameter:

- The ID of the folder, consisting of the path with leading and trailing slashes. Example: "/Projects/Example/".

- Optional: an expiration date and time in ISO 8601 format. Example: "2024-05-15T14:40:35+00:00".

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link loc:item-by-id for the Avid Production Management. The "href" is an RFC-6570 URI template.
 1. Set the URI template variable "id" to the ID of the folder and expand the template.
 2. Add query parameter "?embed=reservations".
 3. Make an HTTP GET request to get the loc:item resource of the folder with an embedded protection:reservations resource.
3. Call HTTP PUT on the embedded protection:set-reservation link with a JSON message body like this for a limited reservation:

None

```
{
  "expiration": "2024-05-15T14:40:35+00:00"
}
Or for an unlimited reservation:
{
  "expiration": null
}
```

To add a reservation to multiple folders, you must run steps 2 and 3 for each folder individually.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Reservations\CTMS Add Reservation.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_avid.ctms.registry -> loc:item-by-id

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources" for "avid-pmplus" systemType, and store the variable globally as "selected_system_locItemByIdHref". Here we also replace "id" parameter in the url with user input "folder_path".

◆ At this step, you need to insert the appropriate **folder_path** in the environment variable.

Get 2_loc:item-by-id -> protection:set-reservation

- Hitting the above fetched 'selected_system_locItemByIdHref', resource of the folder with an embedded protection:reservations resource. Here we need to pass embed='reservations' in query param.
- In the **Scripts** section: From the response, we fetch the "protection:set-reservation", which will be used to add reservation on a given folder, and store it globally as "selected_system_setReservationHref".

PUT 3_Add Reservation

- Hitting the above selected_system_setReservationHref URL, to set the reservation on a folder,
- In the **Body** section: Add request body with the optional "expiration" field to add expiration date and time (if required) for the reservations to be added for the given folderId by hitting the PUT request.

Precautions/Recommendations:

- Please **select and put** the right **folder_id** or **folder_path** in the environment required for this example.

Get Reservation

Summary:

The Interplay Web Services method GetReservations returns reservations on folders and assets.

1. For a folder, it returns the reservations that are directly set on the folder, and reservations inherited from parent folders.
2. For an asset reference in a folder, it can return either the reservations of that specific folder, including the inherited reservations, or it can return the combined reservations of all folders in the folder hierarchy that reference the asset, including the inherited reservations.
3. For an asset referenced just by its MOB ID, it always returns the combined reservations of all folders in the folder hierarchy that reference the asset, including the inherited reservations.

Steps

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link loc:item-by-id for the Avid Production Management. The "href" is an RFC-6570 URI template.
 1. Set the URI template variable "id" to the ID of the folder and expand the URI template.
 2. Add query parameter "?embed=reservations".
 3. Make an HTTP GET request to get the loc:item resource of the folder with an embedded protection:reservations resource containing information about the reservations on the folder.

Example for the embedded protection:reservations resource:

None

```
{
  "reservations": [
    {
      "creator": "svcadmin",
      "expiration": null,    // null means: reservation never
expires
      "inherited": false
    },
    {
      "creator": "svcadmin",
      "expiration": "2024-05-17T14:15:52+00:00",
      "inherited": true
    }
  ]
}
```

The reservations marked with "inherited": true are inherited from a parent folder.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Reservations\CTMS Get Reservation.postman_collection.json", and follow the steps as given below.

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_avid.ctms.registry -> loc:item-by-id

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available within the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources" for "avid-pmplus" systemType, and store the variable globally as "selected_system_folderItemByIdHref" for folder and "selected_system_assetItemByIdHref" for folder that contains asset. Here we also replace "id" parameter in the url with user input "folder_path" to get the reservation on a folder and "asset_path" to get the reservation on a folder that contains an asset.

◆ At this step, you need to insert the appropriate **folder_path** or **asset_path** in the environment variable.

GET 2_Get reservation for a folder in CTMS

- Hitting the above fetched selected_system_folderItemByIdHref, to get the reservations for a folder by id.
- In the **Params** section: Enter the key value pairs as below:
 - embed: reservations

This will fetch you the reservations associated to the given folder in the response.

GET 3_Get reservation for a folder that contains an asset.

- Hitting the above fetched selected_system_assetItemByIdHref, to get the reservations for a folder that contains an asset by assetId.
- In the **Params** section: Enter the key value pairs as below:
 - embed: reservations

- In the **Scripts** section: We fetch "[loc:referencing-items]" link present inside the [loc:referenced-object] in the embedded section. i.e, ["_embedded"]["loc:referenced-object"]["_links"]["loc:referencing-items"], and store the variable globally as "selected_system_referenceltemsHref" to fetch reservation on a referenced asset in subsequent collection.

This will fetch you the reservations for the folder that contains an asset in the response.

GET 4_Get combined reservations of all folders that reference a given asset.

- Hitting the above fetched, "selected_system_referenceltemsHref" to get the reservations for all folders that reference a given asset.
- In the **Params** section: Enter the key value pairs as below:
 - embed: reservations
 - filter: item-type-all

This will fetch you combined reservations of all the folders that reference a given asset in the response. Response contains an embedded array of loc:item resources with all references to the asset in the entire folder hierarchy. Each loc:item resource has an embedded protection:reservations resource with the reservations on the folder.

Precautions/Recommendations:

- Please **select and put** the right **folder_path or asset_path** in the environment required for this example.

Remove Reservation

Summary:

The Interplay Web Services method RemoveReservations removes a direct reservation on a given list of folders. By default, it removes a reservation created by the caller. Optionally, you can remove the reservation that was created by a given user.

Steps

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link loc:item-by-id for the Avid Production Management. The "href" is an RFC-6570 URI template.
 1. Set the URI template variable "id" to the ID of the folder and expand the URI template.
 2. Add query parameter "?embed=reservations".
 3. Make an HTTP GET request to get the loc:item resource of the folder with an embedded protection:reservation resource.

3. To delete the reservation that was created by the current user:
 1. Call HTTP DELETE on the protection:delete-reservation link.
4. To delete the reservation that was created by a given user:
 1. Find the protection:delete-reservation-by-creator link. The "href" is an RFC-6570 URI template.
 2. Set the URI template variable "creator" to the username and expand the URI template.
 3. Call HTTP DELETE on the resulting URI.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Reservations\CTMS Remove Reservation.postman_collection.json" and follow the steps as given below.

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_avid.ctms.registry -> loc:item-by-id

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[loc:item-by-id](#)" link present inside the "resources" for "avid-pmplus" systemType, and store the variable globally as "selected_system_locItemByIdHref". Here we also replace "id" parameter in the url with user input "folder_path".
-  At this step, you need to insert the appropriate **folder_path** in the environment variable.

GET 2_loc:item-by-id -> protection:delete-reservation

- Hitting the above fetched "selected_system_locItemByIdHref", to get the reservations for a folder by id.
- In the **Params** section: Enter the key value pairs as below:
 - embed: reservations
- In the **Scripts** section: We fetch ["protection:delete-reservation"] link present inside the embedded ["protection:reservations"] links for "avid-pmplus" systemType, and store the variable globally as "selected_system_removeReservationHref".

This will fetch you reservations of the given folder. Response contains an embedded array of loc:item resources. Each loc:item resource has an embedded ["protection:reservations"] resource with the reservations on the folder.

DELETE 3_Remove reservation

- Hitting the above fetched "selected_system_removeReservationHref", to remove the reservations for a given folder_path. This will remove the reservations on the given folder path. Response contains empty body with status code as '204 No content'.

Precautions/Recommendations:

- Please **select and put** the right **folder_path** in the environment required for this example.

Documentation Title: Get Resolutions

Summary:

The Interplay Web Services method GetResolutions can be used for two purposes:

- Get the resolutions that are available for a list of masterclips.
 - A caller can get all known resolutions for each masterclip, or only the resolutions that are available online.
 - The method returns a summary of the resolutions for the masterclips and, optionally, the resolutions for each masterclip individually.
- Get the list of known resolutions for the Avid Production Management workgroup.

Get resolutions available for a masterclip in CTMS

Parameter:

- The MOB ID of the asset.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the asset and expand the URI template.

2. Make an HTTP GET request to get the [aa:asset](#) resource for the asset

Make an HTTP GET request to the link `pa:asset-medias` and take the values of the property `"format"` in `"media-items"`. Filter by the `"online"` property if you want to get online resolutions only.

Example:

None

```
{
  "base": {...},
  "common": {...},
  "media-items": [
    {
      "name":
"060a2b340101010101010f0013-000000-e070a775d8eb00a5-241ed6ae5290-
49f1",
      "track": "V1",
      "online": "true",
      "filePath": "//host/.../2024-03-07/202403010v0146ffe522.mxf",
      "type": "filemob",
      "essenceType": "NATIVE",
      "createdBy": "administrator",
      "format": "DV 25 420",
      "size": "7680"
    },
    {
      "name":
"060a2b340101010101010f0013-000000-e170a775d8eb00a5-6ddaf1bbe9eb-
b3a6",
      "track": "A1",
      "online": "true",
      "filePath": "//host/.../2024-03-07/202403010a0146ff5c9a.mxf",
      "type": "filemob",
      "essenceType": "NATIVE",
      "createdBy": "administrator",
      "format": "PCM",
    }
  ]
}
```

```
        "size": "896"
      }
    ],
    "_links": {...}
  }
}
```

In CTMS, you must query the resolutions one-by-one for each masterclip.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Resolutions/CTMS Get Resolutions.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_aaAssetByIdHref`.

◆ At this step, you need to insert the appropriate **master clip ID** in the environment variable named: `masterClipId`

GET 2_aa asset by id

- Hitting the above fetched `selected_system_aaAssetByIdHref`, to get the asset information by id.
- In the **Scripts** section: From the response, we fetch the "pa:asset-medias", which is used to fetch the asset media information that further contains resolutions, and store it globally as `selected_system_paAssetMediasHref`.

GET 3_pa_asset_medias

- Hitting the above selected_system_paAssetMediasHref URL, fetch the resolutions associated to an asset, in this case, the master clip.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Get resolutions for the Avid Production Management workgroup in CTMS

This function is not available in CTMS.

Documentation Title: Restrictions

Add Restrictions in CTMS

Summary:

The Interplay Web Services method AddRestrictions adds restrictions to a masterclip. A restriction is set for a range of frames with a comment describing the restriction.

Parameter:

- The MOB ID of the masterclip.
- An array of restrictions. For each:
 - The start frame (zero-based).
 - The duration in frames.
 - A comment string.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.
 1. Set the URI template variable "id" to the ID of the masterclip and expand the URI template.
 2. Make an HTTP GET request to get the [aa:asset](#) resource for the masterclip.

3. Call HTTP PATCH on the aa:update-time-based link with a JSON message body like this:

None

```
{
  "layers": [
    {
      "name": "restrictions",
      "segments": [
        {
          "id": null,
          "start": 10,
          "duration": 5,
          "attributes": [
            {
              "name": "comment",
              "value": "Access restricted"
            }
          ],
          "userName": "johndoe"
        }
      ]
    }
  ]
}
```

4. The response is a JSON with similar structure and in the same order for the segments where the id property is filled with the generated restriction ID. The request body above contains a layer named "restrictions". Each segment of this layer represents one restriction. It has the following properties:

Segment property	Description
id	The restriction ID. Should be null to create a new restriction. If you use the ID of an existing restriction, the restriction is updated.
start	The start of the restriction (in frames). Same as InFrameNumber in IPWS.

duration	The duration of the restriction (in frames). Same as OutFrameNumber - InFrameNumber + 1 in IPWS.
attributes	Contains an attribute named "comment" with the comment for the segment. Via CTMS, you can also set the "color" attribute of the restriction.
userName	The username for the segment. The value must be given!

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Restrictions/CTMS Add Restrictions.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_aaAssetByldHref`.

◆ At this step, you need to insert the appropriate **master clip ID** in the environment variable named: `masterClipId`

GET 2_aa asset by id

- Hitting the above fetched `selected_system_aaAssetByldHref`, to get the asset by id.
- In the **Scripts** section: From the response, we fetch the "aa:update-time-based", which is used to get time based reference for the given asset, that can be then used to add a restriction, and store it globally as `selected_system_aaUpdateTimeBased`.

PATCH 3 update time based

- Hitting the above `selected_system_aaUpdateTimeBased` URL, to set the asset restrictions.
- In the **Body** section: Add request body with the "layers" and name as restrictions to add restrictions for the given asset by hitting the PATCH request.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.
-

Get Restrictions

Summary:

The Interplay Web Services method GetRestrictions returns time ranges of an asset that have usage restrictions.

In CTMS, you must query the restrictions one-by-one for each masterclip. There is no bulk call for that function.

Parameters

- The MOB ID of the asset.

Steps

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.

1. Set the URI template variable "id" to the ID of the asset and expand the URI template.
2. Add the query parameters "?embed=time-based&layers=restrictions".
3. Make an HTTP GET request to get the [aa:asset](#) resource for the asset with an embedded aa:time-based resource with a layer "restrictions" that contains an array of segments with the restrictions of the asset.

None

```
{
  "assetDuration": 217,
  "editRate": "24000/1001",
  "dropFrame": false,
  "offsetToAssetStart": 0,
  "videoStartTC": "18:17:29:02",
  "_links": { }
```

```

"layers": [
  {
    "name": "restrictions",
    "segments": [
      {
        "id":
"060a2b340101010101010f0013-000000-c39522ac781a4136-bb1dea7ebcf1-
ac77",
        "start": 10,
        "duration": 6,
        "attributes": [
          {
            "name": "comment",
            "value": "Use of swear words"
          },
          {
            "name": "color",
            "value": "WHITE"
          },
          {
            "name": "modified",
            "value": "2024-04-30T14:25:27Z"
          }
        ],
        "userName": "johndoe"
      }
    ]
  }
]
}

```

Mapping of IPWS properties to CTMS properties

IPWS property

CTMS property

Comment	Value of the attribute with name "comment"
InFrameNumber	start : Start frame for the restriction
OutFrameNumber	duration : The duration of the restriction in frames. The last frame of the restriction can be calculated like this: start + duration - 1
Username	userName : Login name of the user that created the restriction.
< not available>	CTMS returns three additional properties: " id ": the ID of the restriction. Attribute " color ": a restriction color that can be set and displayed in Interplay Access. Attribute " modified ": the last modification time of the restriction.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Restrictions/CTMS Get Restrictions.postman_collection.json", and follow the steps as given below.

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as selected_system_aaAssetByIdHref.

◆ At this step, you need to insert the appropriate **master clip ID** in the environment variable named: masterClipId

GET 2_aa asset by id

- Hitting the above fetched selected_system_aaAssetByIdHref, to get the asset by id.
- In the **Params** section: Enter the key value pairs as below:
 - embed: time-based
 - layers: restrictions

This will fetch you the restrictions associated to the given asset in the response.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: References To An Asset (FindLinks in IPWS)

Summary:

The Interplay Web Services method Search is used to search for assets. The method accepts a search request with the following properties:

- **InterplayPathURI (mandatory)**: The folder in which to search for assets.
- **SearchGroup**: The search criteria. See below for details.
- **ReturnAttributes**: a list of attributes to return for each matching asset. If not given, a default set of attributes is returned.
- **MaxResults**: the maximum number of results to return. If not given, all results are returned.

In MediaCentral | Production Management, the SearchGroup parameter allowed for a very fine-granular search with a logical combination with AND, OR, and NOT over conditions on almost all attributes, on categories, on resolutions, and on usage of assets. In Avid Production Management, the search functionality is optimized for performance and supports only conditions for the most frequently used attributes and properties.

The search in Interplay Web Services always returns all references to the matching assets in the given folder and all its sub folders. If a matching asset is in two sub folders, then the result will contain the asset twice, one entry for each of the two sub folders.

In CTMS, there are two different search methods: - **pa:advanced-asset-search**: returns a list of matching assets (masterclips, sequences, sub clips, and so on) in and below the given folder, but without a direct reference to the folder. In CTMS terms, it returns [aa:asset](#) resources. If a matching asset is in two sub folders, it's returned only once in the result set. -

pa:advanced-item-search: returns a list of asset links in folders. In CTMS terms, it returns [loc:item](#) resources. If a matching asset is in two sub folders, the caller can decide with a parameter (oneLinkOnly) if both asset links are returned or only one of them.

Search request in CTMS

The request body is almost the same for pa:advanced-asset-search and pa:advanced-item-search. Example:

None

...

```
{
  "version": "1.0",
  "folder": "/Catalogs/Example",
  "text": "Michael Jordon",
  "type": [ "masterclip", "sequence" ],
  "categories": [ "news", "sports" ],
  "categoriesMode": "anyof",
  "resolution": "PCM",
  "inUse": true,
  "from": "2025-03-06T14:00:00+01:00",
  "to": "2025-03-26T14:33:00+01:00",
  "timeMode": "modified",
  "videoId": "1234",
  "videoIdMode": "exact",
  "multiLink": true,
  "attributes": [
    "com.avid.workgroup.Property.System.FPS",
    "com.avid.workgroup.Property.System.Tracks"
  ],
  "oneLinkOnly": true
}
```

Properties of a search request:

Property	Mandatory	Description
version	mandatory	Version of the search request format. Currently always "1.0".
folder	mandatory	The folder to search in. The method finds assets in the given folder and all its sub folders that the caller can access.

text	optional	Search text. If given, matches only assets that contain the text in the name or comment of the asset.
type	optional	Array of asset types to return. Possible values: "masterclip", "sequence", "subclip", "group", "motioneffect", "effect", "renderedeffect", "stereomasterclip" and "stereosubclip".
categories and categoriesMode	optional	categories is an array of categories to search for. The search behavior depends on the value of categoriesMode: 1. anyof: matches if an asset has any of the given categories. This is the default if categoriesMode is not given. 2. equality: matches if an asset has exactly the given categories and none more. 3. include: matches if an asset has all the given categories and maybe additional ones.
resolution	optional	Resolution to search for. If given, matches only assets that have a track with the given resolution.
inUse	optional	Boolean flag. If given, returns only asset that are used by other assets (inUse=true) or that are not used by other assets (inUse=false).
from, to and timeMode	optional	Defines a time range for the search: 1. timeMode: defines if the time range applies to the creation time (timeMode="created") or the modification time (timeMode="modified"). Default is "created". 2. from: defines the start of the time range. 3. to: defines the end of the time range.
videoid and videoidMode	optional	Searches for assets that match the given videoid. The videoidMode defines the behavior: 1. exact: the Video ID must be exactly the given string. 2. contains: the Video ID must contain the given string.
multiLink	optional	Boolean flag. If given, returns only assets that are in more than one folder (multiLink=true), or only assets that are only in a single folder (multiLink=false).
oneLinkOnly	optional	Boolean flag, only for pa:advanced-item-search. It defines the result for matching assets that are linked in multiple folders: 1. false: returns all links, including multiple links to the same asset in different folders. 2. true: returns exactly one link for each returned asset.
attributes	optional	A list of attributes to return for each asset. If not given, the result contains the values of all attributes.

Search for assets in CTMS

Searches for assets in the given folder and its sub folders that match the given conditions. The result is a HAL resource with embedded [aa:asset](#) links.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link search:searches for the Avid Production Management and call HTTP GET on it.
3. Make an HTTP POST request to the link pa:advanced-asset-search:
 - The request body is a search request as documented above.
 - Optionally, you can set the URI query parameter limit to define a maximum number of results. In any way, the service imposes a maximum number of returned assets to avoid overloading the system.

The response is a HAL resource with: - An optional boolean property `more-results`. It is returned with value "true" if there are more search results than requested with limit or more than the service allows as a maximum number of results. - An embedded list of `[aa:asset](https://developer.avid.com/ctms/api/aa/linkrels/asset.html)` resources with the matching assets. - Each may contain an embedded [aa:attributes](#) resource with either the requested attributes (via the property attributes in the search request) or a default list of attributes.

Example response with one asset (query parameter ?limit=1), abbreviated:

```
None
...
{
  "more-results": true,
  "_links": { ... },
  "_embedded": {
    "aa:asset": [
      {
        "base": {
          "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
          "systemType": "avid-pmplus",
          "id":
            "060a2b340101010101010f0013-000000-1330fb24616e411c-8748d585a9ff-48bd",

```

```

        "type": "masterclip"
      },
      "common": {
        "name": "My test asset",
        ...
      },
      "_links": { ... },
      "_embedded": {
        "aa:attributes": {
          "editRate": "25/1",
          "dropFrame": false,
          "attributes": [
            {
              "name":
"com.avid.workgroup.Property.System.Tracks",
              "value": "V1 A1-2",
              "type": "string",
              "value-labels": {
                "en": "V1 A1-2"
              }
            }
          ],
          "_links": { ... }
        }
      }
    }
  ]
}
}
...

```

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Search/CTMS Search Asset.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "search:searches" link present inside the "resources", and store the variable globally as selected_system_searchSearchesHref.

GET 2 search searches

- Hitting the above fetched selected_system_searchSearchesHref, to get the search link for the given system.
- In the **Scripts** section: From the response, we fetch the "pa:advanced-asset-search", which is used to get the link to search an asset in the system, and store it globally as selected_system_paAdvancedAssetSearchHref.

POST 3 advanced asset search

- Hitting the above selected_system_paAdvancedAssetSearchHref URL, to fetch result of the search URL.

◆ At this step, you need to insert the appropriate **folder, text, type etc.** in the request body.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Search for asset links in CTMS

Searches for asset links in the given folder and its sub folders that match the given conditions. The result is a HAL resource with embedded [loc:item](#) links, which again contain embedded loc:referenced-object links with information about the assets.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link search:searches for the Avid Production Management and call HTTP GET on it.

3. Make an HTTP POST request to the link `pa:advanced-item-search`:
 - The request body is a search request as documented above.
 - Optionally, you can set the URI query parameter `limit` to define a maximum number of results. In any way, the service imposes a maximum number of returned asset links to avoid overloading the system.

The response is a HAL resource with:

- An optional boolean property `more-results`. It is returned with value `"true"` if there are more search results than requested with `limit` or more than the service allows as a maximum number of results.
- An embedded list of [loc:item](#) resources with the matching asset links.
 - Each contains an embedded `loc:referenced-object` resource, which has the exact same structure as an [aa:asset](#) resource, with information about the asset.
 - Each of those again may contain an embedded [aa:attributes](#) resource with either the requested attributes (via the property attributes in the search request) or a default list of attributes.

Example with one asset, abbreviated:

```
None
...
{
  "more-results": true,
  "_links": { ... },
  "_embedded": {
    "loc:item": [
      {
        "base": {
          "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
          "systemType": "avid-pmplus",
          "id":
"/Projects/Example/060a2b34010101010101010f0013-000000-1330fb24616e
411c-8748d585a9ff-48bd",
          "type": "folder-item"
        },
        "common": {
          "name": "My test asset",
          ...
        }
      }
    ]
  }
}
```

```

    "_links": { ... },
    "_embedded": {
      "loc:referenced-object": {
        "base": {
          "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
          "systemType": "avid-pmplus",
          "id":
"060a2b34010101010101010f0013-000000-1330fb24616e411c-8748d585a9ff-
48bd",
          "type": "masterclip"
        },
        "common": {
          "name": "My test asset",
          ...
        },
        "_links": { ... },
        "_embedded": {
          "aa:attributes": {
            "editRate": "25/1",
            "dropFrame": false,
            "attributes": [
              {
                "name":
"com.avid.workgroup.Property.System.Tracks",
                "value": "V1 A1-2",
                "type": "string",
                "value-labels": {
                  "en": "V1 A1-2"
                }
              }
            ],
            "_links": { ... }
          }
        }
      }
    }
  }
}

```

```
}  
]  
  
}  
}  
...
```

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Search/CTMS Search for Asset Link.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "search:searches" link present inside the "resources", and store the variable globally as `selected_system_searchSearchesHref`.

GET 2 search searches

- Hitting the above fetched `selected_system_searchSearchesHref`, to get the search link for the given system.
- In the **Scripts** section: From the response, we fetch the "pa:advanced-item-search", which is used to get the link to search an asset in the system, and store it globally as `selected_system_paAdvancedItemSearchHref`.

POST 3 advanced item search

- Hitting the above `selected_system_paAdvancedAssetSearchHref` URL, to fetch result of the search URL.

◆ At this step, you need to insert the appropriate **folder, text, type etc.** in the request body.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Get Resolutions

Summary:

The Interplay Web Services method CreateShotlist is used to create a simple cuts-only shotlist. The method accepts the following parameters:

- ShotlistName: The name of the created shotlist.
- DestinationFolderURI: The URI of the target folder for the shotlist.
- StartTimecode: The SMPTE timecode of the start of the timeline.
- ShotlistMobID: The MOB ID for the created shotlist. If not given, the service will assign a new MOB ID.
- ShotlistElements: the elements to be put on the timeline of the shotlist. Each element consists of:
 - InterplayURI: the URI of a masterclip or subclip used for the element.
 - A range within the masterclip or subclip:
 - InTimecode: in-point in SMPTE timecode format.
 - OutTimecode: out-point in SMPTE timecode format.
 - Duration: duration of the element.
 - Only one of OutTimecode or Duration should be given.
 - Locators: locators to set, relative to the start of the range.
- Attributes: attributes to set for the shotlist.
- Locators: locators to set, relative to the start of the shotlist timeline.

All masterclips and subclips used in the shotlist must have the same framerate.

- Locators have the following properties:
 - The position of the locator is defined by one of two properties:
 - FrameNumber: frame number, either relative to the start of the shotlist (for the top-level locators), or relative to the start of the element range (for the locators in an element).
 - Timecode: SMPTE timecode for the position of the locator, either on the shotlist timeline (for the top-level locators), or on the masterclip or subclip timeline (for the locators in an element).
 - Comment: comment on the locator.
 - Color: color of the locator.

- Username: creator of the locator. All locators are set on the V1 track of the shotlist.

Creating a shotlist in CTMS

Parameter:

- The ID of the target folder.
- An XMD document with:
 - The name of the shotlist.
 - Optional: the start timecode for the shotlist in SMPTE format (HH:MM:SS:FF).
 - The elements on the timeline. Each element consists of:
 - The asset ID of a masterclip or subclip
 - The range given as in and out points.
 - An optional comment for the element.
 - See below for detailed information about creating an XMD document.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link aa:assets for the Avid Production Management and call HTTP GET on it.
3. Make an HTTP POST request to the link aa:create-xmd-timeline.
 1. Set the query parameter "type" to "shotlist".
 2. Set the query parameter "location" to the ID of the target folder.
 3. Example (with proper URL encoding for the slashes in the folder ID):
?type=shotlist&location=%2FProjects%2FSomeFolder%2F
 4. The request body is the XMD document for the shotlist. See below for details and an example.

The response is a JSON object with two properties, "aa:asset" and "loc:item", both in the same format as the response for an item of a bulk call (see Bulk calls in CTMS for details, here abbreviated):

None

Example:

```
...
{
  "aa:asset": {
    "success": true,
    "data": {
      "base": {
```

```

        "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
        "systemType": "avid-pmplus",
        "id":
"060a2b34010101010101010f0013-000000-0b312000082d00ac-060e2b347f7f-
d080",
        "type": "sequence"
    },
    "common": {
        "name": "New shotlist",
        ...
    },
    "_links": { ... }
},
"loc:item": {
    "success": true,
    "data": {
        "base": {
            "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
            "systemType": "avid-pmplus",
            "id":
"/Catalogs/IPWS//060a2b34010101010101010f0013-000000-0b312000082d00
ac-060e2b347f7f-d080",
            "type": "folder-item"
        },
        "common": {
            "name": "New shotlist",
            ...
        },
        "_links": { ... },
        "_embedded": {
            "loc:referenced-object": { ... }
        }
    }
}
}

```

...

The property [aa:asset](#) contains the information about the created sequence. The format is the same as in the response for bulk calls with the sub properties:

1. success: true if the sequence was created successfully, false if not.
2. data: in case of success, the property contains the [aa:asset](#) resource of the created subclip, including the MOB ID in the property "id" in "base".
3. httpStatus, errorCode and errorMessage: in case of an error.

The property [loc:item](#) contains information about the created asset link in the target folder. The format is the same as above, only with the [loc:item](#) resource of the created sequence in the data property in case of success.

You can use the link aa:update-asset in the returned [aa:asset](#) resource to set common attributes and user attributes of the duplicated asset. See "SetAttributes via CTMS" for examples.

You can use the link aa:update-time-based in the returned [aa:asset](#) resource to set markers. See "SaveUMIDLocators via CTMS" for details. The marker positions in CTMS are positions on the sequence timeline. The locators in the elements in the IPWS call must be re-calculated to the corresponding positions in the shotlist timeline.

Creating an XMD document for a simple shotlist

The XMD document must be in the format as shown in this example:

```
None
{
  "timeline": {
    "version": 4,
    // Name of the shotlist:
    "name": "My new shotlist",
    // Start timecode of the shotlist:
    "stampTimecode": "01:00:00:00",
    "tracks": [
      {
```

```

    "segments": [
      {
        "mob": {
          // asset ID of referenced masterclip or subclip
          "id":
"060a2b340101010101010f0013-000000-6dda7dad08c24cfd-b581d7f05b5b-
51fb"
        },
        "position": {
          "trackIn": -1,
          "trackOut": -1,
          // in and out point of the segment in the
masterclip or subclip
          "in": 5,
          "out": 12
        },
        // comment or title of the segment
        "name": "The referenced master clip."
      },
      ... // more segments in the shotlist
    ]
  }
]
}
}

```

In and out points for segments

In CTMS, the segment ranges use frame numbers instead of SMPTE timecodes as in IPWS.

For segments that reference a masterclip, you can calculate the frame numbers like this:

1. Use [aa:asset-by-id](#) in the CTMS Registry to query the [aa:asset](#) resource for the masterclip.
2. Calculate the difference between the in or out points to the value of the common attribute "startTC".

For subclips, you must re-calculate the in and out points to the original masterclip:

1. Use [aa:asset-by-id](#) in the CTMS Registry to query the [aa:asset](#) resource for the subclip.
2. Call GET on relations:relations with query parameter "?relationtypes=masterclip.forward" to find the ID of the masterclip from which the subclip was created. The result is a relations:relations resource that should contain only a single embedded relation:

None

```
{
  "_embedded": {
    "relations:relation": [
      {
        "relationType": "masterclip.forward",
        "relatedEntity": {
          "id":
"060a2b340101010101010f0013-000000-6dda7dad08c24cfd-b581d7f05b5b-
51fb",
          "kind": "aa",
          "type": "masterclip"
        },
        "_links": {
          "relations:related-entity": {
            "href": "https://..."
          }
        }
      }
    ]
  }
}
```

3. Call GET on the link relations:related-entity to get the [aa:asset](#) resource of the original masterclip.
4. Calculate the difference between the in or out points to the value of the common attribute "startTC" of the masterclip.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Shotlist/CTMS Create Shotlist.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "aa:assets" link present inside the "resources", and store the variable globally as selected_system_aaAssetsHref.

GET 2_aa assets

- Hitting the above fetched selected_system_aaAssetsHref, to get the assets information for the system.
- In the **Scripts** section: From the response, we fetch the "aa:create-xmd-timeline", which is used to create the xmd timeline for creation of shotlist, and store it globally as selected_system_aaCreateXmdTimelineHref.

POST 3 create xmd timeline

- Hitting the above selected_system_aaCreateXmdTimelineHref URL, fetch the resolutions associated to an asset, in this case, the master clip.
- In the **Body** section: Add the request body with "timeline" as mentioned above in the summary tab. Make the POST call to create the shotlist.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: Create Subclip

Summary:

The Interplay Web Services method CreateSubclip is used to create a subclip from a given masterclip. The subclip gets a new MOB ID. The method accepts the following parameters:

- MasterclipURI: (mandatory) The URI of the masterclip.
- The range for the subclip within the masterclip (mandatory):
 - StartingOffset: start frame of the subclip range within the masterclip.

- StartingTimecode: start of the subclip range within the masterclip, given as a SMPTE timecode label.
 - Length: length of the range in frames.
 - Exactly one of StartingOffset and StartingTimecode must be used.
- SubclipPathURI: (mandatory) The URI of the target folder for the created subclip.
- Name: (mandatory) The name of the subclip.
- Attributes: (optional) attributes to set for the subclip.
- Headframe: (optional) the bytes of an image to use as headframe (or thumbnail) for the subclip.

Creating a subclip of a masterclip in CTMS

Parameter:

- The MOB ID of the masterclip.
- The range in the masterclip to use for the sub clip, given as start frame and duration in frames.
- The ID of the target folder.
- Optional: the name of the subclip. If not given, the subclip will get a default name derived from the name of the masterclip.

Steps:

Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.

Find the link [aa:asset-by-id](#) for the Avid Production Management. The "href" is an URI template.

- Set the URI template variable "id" to the ID of the masterclip and expand the URI template.
- Make an HTTP GET request to get the [aa:asset](#) resource for the asset.

Make an HTTP POST request to the link aa:create-subclip with a JSON request body

None

```
{
  "common": {
    "name": "My subclip"           // Name of the subclip,
optional
  },
  "start": 10,                   // Start frame of the subclip
  within the masterclip.
```

```

"duration": 30,                // Duration of the subclip
"location": "/Catalogs/IPWS/"  // ID of the target folder
}

```

The response is a JSON object with two properties, "aa:asset" and "loc:item", both in the same format as the response for an item of a bulk call (see Bulk calls in CTMS for details, here abbreviated):

```

None
{
  "aa:asset": {
    "success": true,
    "data": {
      "base": {
        "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
        "systemType": "avid-pmplus",
        "id":
"060a2b340101010101010f0013-000000-01eacf5d64ed0080-a2083662fe28-
3bc5",
        "type": "subclip"
      },
      "common": {
        "name": "Created subclip",
        ...
      },
      "_links": { ... }
    }
  },
  "loc:item": {
    "success": true,
    "data": {
      "base": {
        "systemID": "E75274DF-BFE6-46CD-BA52-FB58CA0134F2",
        "systemType": "avid-pmplus",

```

```

      "id":
"/Catalogs/IPWS//060a2b340101010101010f0013-000000-01eacf5d64ed00
80-a2083662fe28-3bc5",
      "type": "folder-item"
    },
    "common": {
      "name": "Created subclip",
      ...
    },
    "_links": { ... },
    "_embedded": {
      "loc:referenced-object": { ... }
    }
  }
}

```

The property [aa:asset](#) contains the information about the created asset. The format is the same as in the response for bulk calls with the sub properties:

1. success: true if the asset was created successfully, false if not.
2. data: in case of success, the property contains the [aa:asset](#) resource of the created subclip, including the MOB ID in the property "id" in "base".
3. httpStatus, errorCode and errorMessage: in case of an error.

The property [loc:item](#) contains information about the created asset link in the target folder. The format is the same as above, only with the [loc:item](#) resource of the created subclip in the data property in case of success.

You can use the link [aa:update-asset](#) in the returned [aa:asset](#) resource to set common attributes and user attributes of the subclip. See "SetAttributes via CTMS" for details.

You can use the link [aa:set-thumb-command](#) to set a thumbnail. See "SetHeadframe via CTMS" for details.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "Subclips/CTMS Create Subclip.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_CTMS Registry information: avid.ctms.registry

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as `selected_system_aaAssetByIdHref`.

◆ At this step, you need to insert the appropriate **master clip ID** in the environment variable named: `masterClipId`

GET 2_aa asset by id

- Hitting the above fetched `selected_system_aaAssetByIdHref`, to get the asset information by id.
- In the **Scripts** section: From the response, we fetch the "`aa:create-subclip`", which is used to create a subclip for the selected asset, and store it globally as `selected_system_aaCreateSubclipHref`.

POST 3 create subclip

- Hitting the above `selected_system_aaCreateSubclipHref` URL, we create the subclip for the selected asset.
- In the **Body** section: We provide the subclip details, like the name, the start, duration and location where we need to store this subclip finally.

Precautions/Recommendations:

- Please **select and put** the right asset/masterclip IDs in the environment required for this example.

Documentation Title: UMID Locators

Add and update markers in CTMS

Summary:

The Interplay Web Services method SaveUMIDLocators adds and updates markers (also known as "locators") on an asset. A marker is a marked position on a track of the timeline with a comment and a color.

Parameter:

- The MOB ID of the asset.
- Array of markers to add or update.
 - To update a marker, you must pass in the ID of the marker.
 - To add a marker, omit the ID in the request or set it to null.

Steps:

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link aa:asset-by-id for the Avid Production Management. The "href" is an RFC-6570 URI template.
 1. Set the URI template variable "id" to the ID of the asset and expand the URI template.
 2. Make an HTTP GET request to get the aa:asset resource for the asset.
3. Call HTTP PATCH on the aa:update-time-based link with a request body with the markers to add and update. See below for details and an example. The call returns a similar structure as in the request body, with the created and added segments in the same order but with the new ID of the created segments.

Example request body:

None

```
{
  "layers": [
    {
      "name": "V1",
      "segments": [
        {
          "start": 2924,
          "duration": 1,
```

```

        "attributes": [
            {
                "name": "comment",
                "value": "Lionel Messi scores a goal"
            },
            {
                "name": "color",
                "value": "Blue"
            }
        ],
        "userName": "janesmith"
    },
    {
        "id":
"060a2b34010101010101010f0013-000000-b238acb69d8e412b-9a0d648212d4-
cae4",
        "start": 3724,
        "duration": 1,
        "attributes": [
            {
                "name": "comment",
                "value": "Substitution: Andrés Iniesta for Lionel
Messi"
            },
            {
                "name": "color",
                "value": "Black"
            }
        ],
        "userName": "janesmith"
    }
]
}
]
}

```

The request body is an aa:time-based resource. It contains a property "layers" with an array of layers. In Avid Production Management, the "name" of the layer is the name of the track ("V1", "A1", "A2", ...) for the markers. The property "segments" contains the markers in the form of segments.

A marker is a segment with the following properties:

Segment property	Description
id	The segment ID. SSet it to an existing marker ID to update the marker, or to null to add a new marker.
start	The position of the marker. The value is a zero-based frame number within the video.
duration	Always 1. Markers are set on a single frame.
attributes	Array of attributes to set for the marker: "comment": the text of the marker , "color": the color of the marker.
userName	The login name of the user that created a marker. The property is mandatory and must be set for Avid Production Management.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "UMID Locators\CTMS Add-Update Markers.postman_collection.json", and follow the steps as given below:

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project-level ReadMe to understand how it works.)

GET 1_avid.ctms.registry -> getServiceRoots (set assetByld from global var asset_id)

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as "selected_system_assetByldHref". Here we also replace "id" parameter in the url with user input "asset_id".

- ◆ At this step, you need to insert the appropriate **asset_id** in the environment variable.

Get 2_asset-by-id to update-time-based

- Hitting the above fetched "selected_system_assetByIdHref" url to get the link relation for "[aa:update-time-based]". Here we need to pass embed='time-based' in query param.
- In the **Scripts** section: From the response, we fetch the "[aa:update-time-based]", and store the variable globally as "selected_system_updateTimeBasedHref".

PATCH 3_Add/Update Markers

- Hitting the above selected_system_updateTimeBasedHref URL, to add/update the markers on a layer in a given asset.
- In the **Body** section: Add request body with the layer name, segment id and segment attributes to be updated. If segmentId is not passed, then a new segment/marker with new segmentId is created under same layer.

Response contains a similar structure as in the request body, with the created and added segments in the same order but with the new ID of the created segments.

Precautions/Recommendations:

- Please **select and put** the right **asset_id** in the environment required for this example.
-

Get markers in CTMS

Summary:

The Interplay Web Services method GetUMIDLocators returns the markers (also known as "locators") of an asset. A marker is a marked position on a track of the timeline with a comment and a color.

Parameter:

- The MOB ID of the asset.

Steps

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link aa:asset-by-id for the Avid Production Management. The "href" is an RFC-6570 URI template.
 1. Set the URI template variable "id" to the ID of the asset and expand the URI template.

2. Add the query parameter "?embed=time-based".
3. Make an HTTP GET request to get the aa:asset resource for the asset with an embedded aa:time-based resource.

Example (abbreviated):

None

```
{
  "editRate": "24000/1001",
  "dropFrame": false,
  "offsetToAssetStart": 0,
  "videoStartTC": "18:17:29:02",
  "_links": { ... },
  "layers": [
    {
      "name": "V1",
      "segments": [
        {
          "id":
"060a2b34010101010101010f0013-000000-b238acb69d8e412b-9a0d648212d4-
cae4",
          "start": 45,
          "duration": 1,
          "attributes": [
            {
              "name": "color",
              "value": "Green"
            },
            {
              "name": "comment",
              "value": "test comment 2: 2024-04-30 10:10:16.291"
            }
          ],
          "userName": "svcqa"
        }
      ]
    }
  ]
}
```

```
]
}
```

The resource may also contain the layer "restrictions" with restrictions on the asset. This layer can be ignored here.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "UMID Locators\CTMS Get Markers.postman_collection.json", and follow the steps as given below.

Postman API Calls

(Hit Send on the "ROPC Login" API call to login in the system. View the project's ReadMe to understand how it works.)

GET 1_avid.ctms.registry -> getServiceRoots (set assetById from global var asset_id)

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as "selected_system_assetByIdHref". Here we also replace "id" parameter in the url with user input "asset_id".

◆ At this step, you need to insert the appropriate **asset_id** in the environment variable.

Get 2_asset-by-id -> aa:time-based

- Hitting the above fetched "selected_system_assetByIdHref" url to get the link relation for "[aa:time-based]". Here we need to pass embed='time-based' in query param.
- In the **Scripts** section: From the response, we fetch the "[aa:time-based]", and store the variable globally as "selected_system_timeBasedHref".

GET 3_Get Markers

- Hitting the above "selected_system_timeBasedHref" URL, to get all markers on a layer V1 on a given asset.

Response contains the layers with names like "V1", "V2", "A1", "A2" for each track of the asset that has markers. The property "segments" contains the information about the markers on each track.

Precautions/Recommendations:

- Please **select and put** the right **asset_id** in the environment required for this example.
-

Remove Markers in CTMS

Summary:

The Interplay Web Services method RemoveUMIDLocators removes markers (also known as "locators") from an asset. A marker is a marked position on a track of the timeline with a comment and a color.

Parameter:

- The MOB ID of the asset.
- An array of marker IDs.

Steps

1. Use the cached CTMS Registry information or fetch it as shown in Using the CTMS API.
2. Find the link aa:asset-by-id for the Avid Production Management. The "href" is an RFC-6570 URI template.
 1. Set the URI template variable "id" to the ID of the asset and expand the URI template.
 2. Make an HTTP GET request to get the aa:asset resource for the asset.
3. Find the aa:delete-time-based link in the aa:asset resource.
 1. Add query parameter "segmentids" with a comma-separated list of marker IDs you want to delete.
 2. Call HTTP DELETE on the resulting URI to delete the segments with the given IDs in all layers.

How to use this collection:

This collection involves the process of following the above steps, as provided in the summary, in your postman application. Open your postman, import the collection "UMID Locators\CTMS Remove UMID Locators.postman_collection.json" and follow the steps as given below.

Postman API Calls

(Hit Send on the "ROPC Login" API call to get logged in the system. View the project's ReadMe to understand how it works.)

GET 1_avid.ctms.registry -> getServiceRoots (set assetByld from global var asset_id)

- This GET call hits the service roots URL for CTMS registry, fetching the resources, systems and the links associated to the system.
- In the **Scripts** section: We check if the Avid Production Management is available with the registry. If yes, we fetch "[aa:asset-by-id](#)" link present inside the "resources", and store the variable globally as "selected_system_assetByldHref". Here we also replace "id" parameter in the url with user input "asset_id".

◆ At this step, you need to insert the appropriate **asset_id** in the environment variable.

Get 2_asset-by-id -> delete-time-based

- Hitting the above fetched "selected_system_assetByldHref" url to get the link relation for "[aa:delete-time-based]". Here we need to pass embed='time-based' in query param.
- In the **Scripts** section: From the response, we fetch the "[aa:delete-time-based]", and store the variable globally as "selected_system_delete-time-based".

DELETE 3_Delete Markers

- Hitting the above "selected_system_delete-time-based" URL, to delete the segments(markers) with the given IDs in all layers(tracks).
- Add query parameter "segmentids" with a comma-separated list of marker IDs you want to delete.

◆ At this step, you need to be careful about inserting the appropriate **segmentids** in the parameter. If **segmentids** is passed as null or not passed, it will delete all the markers and restrictions on given asset.

None

Markers in MediaCentral | Production Management and MediaCentral | Avid Production Management have unique IDs across all layers of time-based information. That's why it is sufficient to filter by segment IDs, meaning that the segments with the given IDs are deleted from all layers. For other products like, for example,

MediaCentral | Asset Management, the segment IDs are only unique within a layer, but not across layers. A different segment in a different layer could have the same ID.

In that case you must combine the "segmentids" query parameter with the "layers" query parameter to identify a specific segment in a specific layer. **

Precautions/Recommendations:

- Please **select and put** the right **asset_id** in the environment required for this example.
- Make sure that you always provide the query parameter **segmentids** with a non-empty list of marker IDs! The query parameters for the aa:delete-time-based link work like a filter for the segments to be deleted. **If you omit or mistype the query parameter, you will delete all markers and restrictions.**